DV INTERVIEW · GLANCE SHEET

AXI Protocol: Channels, Handshake & Ordering

AMBA Advanced eXtensible Interface — 5 independent channels, burst-based, ID-tagged, multiple outstanding transactions.

Topic: Intro to AXI · AXI channels · In-order vs out-of-order transactions

Golden rule → transfer happens only when VALID & READY are both HIGH at a rising clock edge

1 Where AXI sits in AMBA

Bus	Use	Traits
APB	Slow peripherals (UART, GPIO, timers)	No pipelining, no burst, simple setup→access
AHB	Mid-performance system bus	Single shared channel, pipelined addr/data, one master at a time
AXI	High-performance SoC fabric (CPU, DDR, DMA, GPU)	5 separate channels, bursts, IDs, outstanding & out-of-order
AXI-Lite	Register / control access	Single-beat only, no IDs, no bursts
AXI-Stream	Data pipes (video, DSP, network)	No address; TVALID/TREADY/TDATA/TLAST

Terminology: **Master / Manager** initiates, **Slave / Subordinate** responds. An interconnect connects many of each.

2 Why AXI is fast

- **Separate address/control and data phases** — address can be sent ahead of data.
- **Separate read and write channels** — reads and writes run simultaneously (full duplex).
- **Burst transactions** — only the *start address* is sent; slave computes the rest.
- **Multiple outstanding transactions** — issue new addresses before earlier ones complete.
- **Out-of-order completion** using transaction IDs.
- **Unaligned transfers & byte lanes** via `WSTRB`.
- **Register slices** can be inserted on any channel for timing closure (channels are independent).

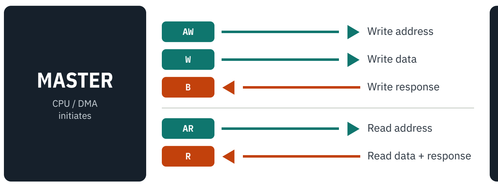
One-liner: AXI decouples address, data and response so latency is hidden by pipelining and parallelism.

3 AXI3 vs AXI4 (asked a lot)

Feature	AXI3	AXI4
INCR burst length	1–16	**1–256**
AxLEN width	4 bits	8 bits
Write interleaving	Allowed (has `WID`)	**Not allowed** — WID removed
Locked transfers	Locked + exclusive	Exclusive only (AxLOCK 1 bit)
New signals	—	`AxQOS`, `AxREGION`, `xUSER`
WRAP / FIXED length	WRAP 2/4/8/16; FIXED ≤16	Same (only INCR grew)

AXI4 write response may only come after **both** AW and the last W beat are handshaken.

4 The 5 channels — each has its own VALID / READY



Ch	Key signals	Note
AW	`AWID AWADDR AWLEN AWSIZE AWBURST`	One per burst
W	`WDATA WSTRB WLAST`	One per beat; WLAST on final beat
B	`BID BRESP`	One per *burst*, not per beat
AR	`ARID ARADDR ARLEN ARSIZE ARBURST`	One per burst
R	`RID RDATA RRESP RLAST`	RRESP on *every* beat

Write = 3 channels (AW, W, B). **Read = 2 channels** (AR, R) — read response rides on the R channel, so there's no separate read-response channel.

Direction: AW, W, AR driven by master; B and R driven by slave. READY always flows opposite to VALID.

Why no separate read response? Data already returns slave→master, so RRESP is attached to each R beat for free.

5 VALID / READY handshake

- **Source** drives VALID (info available); **destination** drives READY (can accept).
- Transfer occurs on the rising edge where **VALID = 1 and READY = 1**.
- **Once VALID is asserted it must stay HIGH**, with data/control stable, until the handshake completes.
- **Source must NOT wait for READY** before asserting VALID (else deadlock).
- **Destination MAY wait for VALID** before asserting READY; READY may also be asserted early and de-asserted before VALID.

THREE LEGAL ORDERINGS (TRANSFER CYCLE IN ORANGE)

CLOCK	T1	T2	T3	T4	T5
a) VALID first · VALID	0	1	1	1	0
READY	0	0	0	1	0
b) READY first · VALID	0	0	0	1	0
READY	0	1	1	1	0
c) Same cycle · VALID	0	0	1	0	0
READY	0	0	1	0	0

Interview trap: "Can VALID drop before READY comes?" → No, that's a protocol violation. "Can READY drop without VALID?" → Yes.

6 Channel dependencies

WRITE TRANSACTION

`AW handshake` + `W … WLAST handshake` → `B response`

- W data **may arrive before** AW (slave may wait for both).
- Slave must wait for **WLAST** (and in AXI4, AW too) before asserting BVALID.

READ TRANSACTION

`AR handshake` → `R beats … RLAST`

- Slave must wait for **ARVALID & ARREADY** before asserting RVALID.

DEADLOCK-AVOIDANCE RULES

- Slave may wait for AWVALID/WVALID before AWREADY/WREADY.
- Master must **not** wait for AWREADY/WREADY before driving W/AW VALID.
- Master may wait for BVALID before BREADY (but it's good practice to keep BREADY/RREADY high).

Count rule: #W beats per burst = AWLEN + 1, WLAST exactly on the last one. #R beats = ARLEN + 1, RLAST on the last.

7 Bursts: LEN · SIZE · TYPE

AxLEN
beats − 1
`beats = LEN + 1`

AxSIZE
bytes per beat
`bytes = 2^SIZE` (≤ bus width)

Total bytes
`(LEN+1) × 2^SIZE`

AxBURST	Type	Address behaviour	Typical use
`2'b00`	**FIXED**	Same address every beat	FIFO / peripheral register
`2'b01`	**INCR**	Address += 2^SIZE each beat	Normal memory access
`2'b10`	**WRAP**	Increments, then wraps at boundary	Cache-line fill (critical word first)
`2'b11`	Reserved	—	Flag as error in checker

4 KB rule: a burst must never cross a 4 KB address boundary (a slave region is ≥ 4 KB, so a burst stays in one slave).

WRAP rules: length 2, 4, 8 or 16 only; start address must be aligned to transfer size.

WORKED WRAP EXAMPLE

WRAP, 4 beats, 4 bytes/beat, start `0x38` → container = 4 × 4 = 16 B → lower boundary = `INT(0x38/16)×16 = 0x30`, upper = 0x40

`0x38` → `0x3C` → wrap → `0x30` → `0x34`

Same with **INCR**: 0x38 → 0x3C → 0x40 → 0x44. With **FIXED**: 0x38 ×4.

8 Responses & side-band signals

xRESP	Name	Meaning
`00`	**OKAY**	Normal success (or exclusive access failed)
`01`	**EXOKAY**	Exclusive access succeeded
`10`	**SLVERR**	Slave reached, but it reported an error
`11`	**DECERR**	Decode error — no slave at that address (interconnect default slave)

Signal	Purpose
`WSTRB`	1 bit per data byte; which lanes are valid (narrow/unaligned writes)
`AxLOCK`	Exclusive access (atomic read-modify-write semaphores)
`AxCACHE`	Bufferable, modifiable, allocate hints
`AxPROT`	Privileged / secure / instruction vs data
`AxQOS`	Priority hint for arbitration (AXI4)
`AxREGION`	Multiple logical regions in one slave (AXI4)
`xUSER`	User-defined side-band (AXI4)

WSTRB example (32-bit bus): write 1 byte at 0x1002 → `WSTRB = 4'b0100`. Narrow transfer: SIZE < bus width, active lanes rotate each beat.

9 Transaction IDs: in-order vs out-of-order

THE RULE

Same ID → must complete in issue order.
Different IDs → may complete in any order.

- ID = "ordering tag", typically one per master thread / stream.
- The interconnect **extends the ID** with master-port bits so responses route back to the right master.
- No ordering between reads and writes, even with the same ID — master must wait for a response if it needs that.

EXAMPLE — MASTER ISSUES 3 READS

Issued: `A · ID0` `B · ID1` `C · ID0`

Legal: `B` `A` `C` ✓ B overtakes (diff ID)

Legal: `A` `B` `C` ✓ in-order

Illegal: `C` `A` `B` × C before A (same ID0)

Why out-of-order? A fast slave (SRAM) shouldn't be blocked by a slow slave (DDR). Different IDs let the fast response return first.

OUTSTANDING & INTERLEAVING

- **Outstanding:** master sends next AR/AW before previous data/response returns. Depth = how many in flight (read/write acceptance capability).
- **Read data interleaving:** beats of bursts with *different* RIDs may be mixed; within one burst, beats stay in order.
- **Write data interleaving:** AXI3 only (WID). **AXI4: W beats must follow AW order**, no interleaving.
- In-order slaves simply return everything in issue order — always legal.

Mode	Pros	Cons
In-order	Simple slave & master	Head-of-line blocking
Out-of-order	Higher throughput, lower latency	Reorder buffers, ID tracking

10 How a DV engineer verifies AXI

TESTBENCH (UVM AXI VIP)

- Agent per interface: **sequencer · driver · monitor**; separate drivers/threads per channel so channels stay independent.
- Monitor rebuilds bursts from beats; sends to scoreboard via analysis port.
- **Scoreboard:** reference memory model + **per-ID queues** (assoc array keyed by ID) → checks same-ID order, allows cross-ID reordering.
- Randomize READY delays (back-pressure) and VALID delays.

SVA PROTOCOL CHECKS

- VALID stays high & payload stable until READY: `valid && !ready |=> valid && $stable(payload)`
- No X on VALID after reset; VALIDs low during reset.
- WLAST/RLAST on beat LEN+1 exactly; no beat after LAST.
- No 4 KB crossing; WRAP len ∈ {2,4,8,16}; WRAP addr aligned; BURST ≠ 2'b11.
- BVALID only after AW + WLAST; RVALID only after AR; BID/RID match an outstanding ID.

FUNCTIONAL COVERAGE

- Cross **BURST × LEN × SIZE**, aligned vs unaligned, all WSTRB patterns.
- All 4 responses, outstanding depth (0 … max), same-ID vs diff-ID reorder seen.
- Handshake orderings a/b/c, back-to-back transfers, reads & writes concurrently.

11 Rapid-fire interview Q&A

Question	Crisp answer
How many channels? Why separate?	5 (AW, W, B, AR, R); independent = parallel R/W, pipelining, register slices.
Why no read-response channel?	RRESP travels with each R beat.
Max burst length?	AXI4 INCR 256; AXI3 16; WRAP 16; FIXED 16.
Why the 4 KB boundary?	Min slave address region; keeps a burst inside one slave & one page.
Can W come before AW?	Yes; slave may buffer it.
Purpose of WLAST if LEN is known?	Slave needn't count beats; simplifies logic & checking.
Why was WID removed in AXI4?	Write interleaving was rarely used and complicated slaves.
How is out-of-order achieved?	Different IDs; same-ID responses stay in order.
SLVERR vs DECERR?	SLVERR from the slave; DECERR from interconnect when no slave decodes.
Who can deadlock and how?	Source waiting for READY before VALID — forbidden.
AXI vs AHB in one line?	AHB: one shared pipelined bus; AXI: split channels, outstanding, OoO.
Bytes in LEN=7, SIZE=3?	8 beats × 8 B = 64 B.

DV INTERVIEW · GLANCE SHEET 2

AXI Signals, Channel by Channel

Every signal on the read and write channels: who drives it, how wide it is, and what it means.

`M→S` driven by master
`S→M` driven by slave

Naming trick: prefix = channel (AW, W, B, AR, R)
Ax* = same signal on both AW and AR

G Global signals & reset rules

Signal	Source	Description
ACLK	`Clock`	Single global clock; all inputs sampled and outputs changed on its **rising edge**.
ARESETn	`Reset`	Global reset, **active LOW**. Can assert asynchronously; must de-assert synchronously to ACLK.

- During reset: master drives `ARVALID`, `AWVALID`, `WVALID` LOW; slave drives `RVALID`, `BVALID` LOW.
- VALIDs may go HIGH only at a rising edge **after** ARESETn is HIGH.
- All other signals can be any value during reset.

No combinational path allowed from input to output on a master or slave interface (avoids loops across the interconnect).

⇄ Signal groups on every address channel

Identity
`AxID` — ordering tag

Where
`AxADDR` — start byte address

How much
`AxLEN` beats, `AxSIZE` bytes/beat

Pattern
`AxBURST` — FIXED / INCR / WRAP

Attributes
`AxLOCK AxCACHE AxPROT`

AXI4 extras
`AxQOS AxREGION AxUSER`

Then the handshake pair: `AxVALID` (M→S) and `AxREADY` (S→M). Learn the AW list once — **AR is an exact mirror**.

AW Write address channel

Signal	Width	Src	Description
AWID	impl.	`M→S`	ID tag for the write transaction
AWADDR	32/64…	`M→S`	Address of the first transfer in the burst
AWLEN	8 (AXI3: 4)	`M→S`	Number of beats − 1
AWSIZE	3	`M→S`	Bytes per beat = 2^AWSIZE
AWBURST	2	`M→S`	Burst type: FIXED / INCR / WRAP
AWLOCK	1 (AXI3: 2)	`M→S`	Normal or exclusive access
AWCACHE	4	`M→S`	Bufferable / cacheable / allocate attributes
AWPROT	3	`M→S`	Privilege, security, instruction/data
AWQOS	4	`M→S`	Quality-of-service priority (AXI4)
AWREGION	4	`M→S`	Region selector within a slave (AXI4)
AWUSER	impl.	`M→S`	User-defined sideband (AXI4)
AWVALID	1	`M→S`	Address & control are valid
AWREADY	1	`S→M`	Slave can accept the address

W Write data channel

Signal	Width	Src	Description
WID	impl.	`M→S`	**AXI3 only** — ID of the write data (enables interleaving)
WDATA	8…1024	`M→S`	Write data; width = 8, 16, 32 … 1024 bits
WSTRB	DATA/8	`M→S`	Byte strobes — 1 bit per byte lane, HIGH = lane valid
WLAST	1	`M→S`	Marks the last beat of the burst
WUSER	impl.	`M→S`	User sideband (AXI4)
WVALID	1	`M→S`	Write data & strobes are valid
WREADY	1	`S→M`	Slave can accept the data

No address on W — the slave links data to the address by **order** (AXI4) or by **WID** (AXI3).

B Write response channel

Signal	Width	Src	Description
BID	impl.	`S→M`	Must equal AWID of the transaction being answered
BRESP	2	`S→M`	Status of the whole write burst
BUSER	impl.	`S→M`	User sideband (AXI4)
BVALID	1	`S→M`	Write response is valid
BREADY	1	`M→S`	Master can accept the response

One B per burst, not per beat. Slave sends it only after the last W beat (WLAST).

AR Read address channel

Signal	Width	Src	Description
ARID	impl.	`M→S`	ID tag for the read transaction
ARADDR	32/64…	`M→S`	Start address of the read burst
ARLEN	8 (AXI3: 4)	`M→S`	Number of beats − 1
ARSIZE	3	`M→S`	Bytes per beat = 2^ARSIZE
ARBURST	2	`M→S`	FIXED / INCR / WRAP
ARLOCK	1 (AXI3: 2)	`M→S`	Normal or exclusive
ARCACHE · ARPROT	4 · 3	`M→S`	Memory & protection attributes
ARQOS · ARREGION · ARUSER	4 · 4 · impl.	`M→S`	AXI4 additions
ARVALID	1	`M→S`	Read address & control valid
ARREADY	1	`S→M`	Slave can accept the address

R Read data channel

Signal	Width	Src	Description
RID	impl.	`S→M`	Must equal the ARID it is answering
RDATA	8…1024	`S→M`	Read data
RRESP	2	`S→M`	Status of **this beat** (can differ beat to beat)
RLAST	1	`S→M`	Last beat of the read burst
RUSER	impl.	`S→M`	User sideband (AXI4)
RVALID	1	`S→M`	Read data is valid
RREADY	1	`M→S`	Master can accept the data

Read has **no strobe** — the master just ignores bytes it doesn't need. Slave must return all LEN+1 beats even if an error occurs.

E Encodings to memorise

AxSIZE	Bytes/beat
3'b000	1
3'b001	2
3'b010	4
3'b011	8
3'b100	16
3'b101	32
3'b110	64
3'b111	128

AxBURST	Type
2'b00	FIXED
2'b01	INCR
2'b10	WRAP
2'b11	Reserved

xRESP	Meaning
2'b00	OKAY
2'b01	EXOKAY
2'b10	SLVERR
2'b11	DECERR

AxPROT bit	0 / 1
[0]	Unprivileged / Privileged
[1]	Secure / Non-secure
[2]	Data / Instruction

AxLOCK	Access
0	Normal
1	Exclusive
AXI3 2'b10	Locked (dropped in AXI4)

AxCACHE bit	Meaning
[0]	Bufferable
[1]	Modifiable (AXI3: Cacheable)
[2]	Read-allocate
[3]	Write-allocate

Quick math: AxSIZE must not exceed the data bus width. 64-bit bus → max SIZE = 3'b011 (8 bytes).

→ Read & write flows, step by step

WRITE (3 CHANNELS)

`1 · AWVALID/AWREADY` → `2 · WVALID/WREADY × (LEN+1), WLAST` → `3 · BVALID/BREADY + BRESP`

- Master puts address/control on AW and raises AWVALID; slave takes it with AWREADY.
- Master sends each beat with WDATA + WSTRB; WLAST on the final one.
- Slave returns one BRESP with BID = AWID.

READ (2 CHANNELS)

`1 · ARVALID/ARREADY` → `2 · RVALID/RREADY × (LEN+1), RLAST, RRESP each beat`

- Master puts address on AR; slave accepts with ARREADY.
- Slave returns data beats with RID = ARID, RRESP per beat, RLAST on the last.

Example: 32-bit bus, read 16 bytes from 0x100 INCR → ARLEN = 3, ARSIZE = 3'b010, ARBURST = 2'b01 → beats at 0x100, 0x104, 0x108, 0x10C.

? Signal-level interview Q&A

Question	Crisp answer
Which signals does the slave drive?	All READYs on AW/W/AR, plus everything on B and R except BREADY/RREADY.
Why does W have no address?	Address was already sent on AW; slave matches by order (or WID in AXI3).
Why no RSTRB?	Master simply discards unused read bytes; reads have no side effect on lanes.
WSTRB width for 128-bit data?	16 bits (one per byte).
Is RRESP per burst or per beat?	Per beat. BRESP is per burst.
What must BID / RID equal?	The AWID / ARID of the transaction being answered.
Can WSTRB be all zeros?	Yes — a legal beat that writes no bytes.
Reset value of VALIDs?	LOW; raised only after ARESETn goes HIGH.
Signals removed in AXI4?	WID; locked transfers (AxLOCK reduced to 1 bit).
Signals added in AXI4?	AxQOS, AxREGION, xUSER; AxLEN widened to 8 bits.

DV INTERVIEW · GLANCE SHEET 3

AXI Write & Read Transfers, Cycle by Cycle

How a burst actually moves on the wires: handshakes, stalls, last beats, responses and byte lanes.

1 signal HIGH
⇄ handshake (transfer happens)
0 signal LOW / idle

W Write burst — 4 beats, INCR, 32-bit bus, AWADDR = 0x00

`AWLEN = 3` · `AWSIZE = 3'b010` (4 B) · `AWBURST = 2'b01` · slave stalls the write data once (WREADY low in T4).

ACLK	T1	T2	T3	T4	T5	T6	T7	T8	T9	T10
AWVALID	1	⇄	0	0	0	0	0	0	0	0
AWREADY	0	⇄	0	0	0	0	0	0	0	0
AWADDR	0x00	0x00	–	–	–	–	–	–	–	–
WVALID	0	0	⇄	1	⇄	⇄	⇄	0	0	0
WREADY	0	0	⇄	0	⇄	⇄	⇄	0	0	0
WDATA	–	–	D0	D1	D1	D2	D3	–	–	–
WLAST	0	0	0	0	0	0	1	0	0	0
BVALID	0	0	0	0	0	0	0	1	⇄	0
BREADY	0	0	0	0	0	0	0	0	⇄	0
BRESP	–	–	–	–	–	–	–	OKAY	OKAY	–

T2 — address accepted (AWVALID & AWREADY).

T3, T5, T6, T7 — 4 data beats. **T4** = slave back-pressure: master **holds D1** and WVALID until accepted.

T7 — WLAST with beat 4 (= AWLEN + 1).

T8–T9 — slave raises BVALID after WLAST; master accepts at T9 with BREADY → write done.

R Read burst — 4 beats, INCR, 32-bit bus, ARADDR = 0x40

`ARLEN = 3` · `ARSIZE = 3'b010` · `ARBURST = 2'b01` · the **master** stalls once (RREADY low in T6).

ACLK	T1	T2	T3	T4	T5	T6	T7	T8	T9	T10
ARVALID	⇄	0	0	0	0	0	0	0	0	0
ARREADY	⇄	0	0	0	0	0	0	0	0	0
ARADDR	0x40	–	–	–	–	–	–	–	–	–
RVALID	0	0	0	⇄	⇄	1	⇄	⇄	0	0
RREADY	1	1	1	⇄	⇄	0	⇄	⇄	1	1
RDATA	–	–	–	D0	D1	D2	D2	D3	–	–
RRESP	–	–	–	OK	OK	OK	OK	OK	–	–
RLAST	0	0	0	0	0	0	0	1	0	0

T1 — address accepted in the same cycle (VALID & READY together).

T2–T3 — slave latency; RREADY can sit HIGH waiting (allowed).

T6 — master back-pressure: slave **holds D2** and RVALID until RREADY returns.

T8 — RLAST on beat 4 (addr 0x4C). No separate response channel — RRESP came with every beat.

1 Single-beat transfer (AWLEN = 0)

- One AW handshake, one W beat with **WLAST = 1**, one B response.
- AW and W can handshake in the **same cycle**, or W can even come **first**.
- Fastest write = 2 cycles (AW+W together, then B). Fastest read = AR, then R next cycle.
- AXI-Lite is always this case — no LEN, no LAST, no ID.

WRITE ORDERINGS THE SLAVE MUST HANDLE

AW then W then B — normal

AW + W same cycle then B — fastest

W then AW then B — legal, often missed in TBs

2 Overlapping (outstanding) transfers

Master issues the next address before the previous burst's data finishes — this hides slave latency.

Cycle	T1	T2	T3	T4	T5	T6	T7	T8
AR	A	B	–	–	–	–	–	–
R	–	–	A0	A1	A2	A3·L	B0	B1·L

- Without overlap B's address would wait until T7 → 4 wasted cycles.
- Same ID → B's data must follow A's. Different IDs → B's data could return first.
- Max outstanding is a design parameter ("acceptance capability") — cover 0 … max in DV.

3 Byte lanes: narrow & unaligned transfers (32-bit bus = 4 lanes)

NARROW INCR · SIZE = 1 B · START 0X00 · 4 BEATS

	B3	B2	B1	B0	
beat 1				B0	0001
beat 2			B1		0010
beat 3		B2			0100
beat 4	B3				1000

Narrow = SIZE smaller than bus. Active lane **moves** each beat with the address.

UNALIGNED INCR · SIZE = 4 B · START 0X01 · 3 BEATS

	B3	B2	B1	B0	
0x01	B3	B2	B1		1110
0x04	B3	B2	B1	B0	1111
0x08	B3	B2	B1	B0	1111

Only the **first beat** is unaligned; later beats snap to aligned addresses (0x04, 0x08). Total = 11 bytes.

RULES TO REMEMBER

- Lane for a byte = `addr mod (bus bytes)` (little-endian).
- INCR next address = `aligned(addr) + 2^SIZE`.
- **FIXED** narrow: the same lanes every beat.
- **WRAP** must start aligned — unaligned WRAP is illegal.
- Reads use the same lane logic; master just picks the bytes it needs.
- WSTRB must be 0 on lanes outside the transfer.

4 What to check & cover for transfers

CHECKS (MONITOR / SVA)

- Payload (WDATA, WSTRB, RDATA…) **stable while VALID && !READY** — the T4 / T6 stall cases above.
- Exactly LEN+1 beats; LAST only on the final beat.
- BVALID never before WLAST handshake; RVALID never before AR handshake.
- WSTRB lanes match address/SIZE for narrow & unaligned beats.
- Read data matches reference memory; write updates only strobed bytes.

COVERAGE

- Back-pressure on every channel (READY low for 0, 1, N cycles).
- W before AW, same-cycle AW+W, AW before W.
- Back-to-back bursts (no idle), overlapped addresses, max outstanding.
- Narrow × aligned/unaligned × burst type × LEN.
- Error responses mid-burst on R (RRESP changes beat to beat).

? Transfer interview Q&A

Question	Crisp answer
Minimum cycles for a single write?	2 — AW+W handshake together, B next cycle.
What happens when WREADY drops mid-burst?	Master holds the same WDATA/WSTRB and WVALID until WREADY returns.
Can RREADY be held HIGH all the time?	Yes, and it's common; it just means "always ready".
How does the slave know the burst ended?	WLAST (it may also count LEN+1).
Can a read error stop the burst early?	No — all LEN+1 beats are still returned with RRESP error.
Start 0x03, SIZE 4 B, 2 beats — strobes?	Beat 1: 1000, beat 2 (0x04): 1111.
Why overlap addresses?	Hides latency; keeps the data channel busy back-to-back.
Can AW for burst 2 come before W of burst 1 ends?	Yes — outstanding writes; in AXI4 W data still follows AW order.
Who drives BREADY / RREADY?	The master.

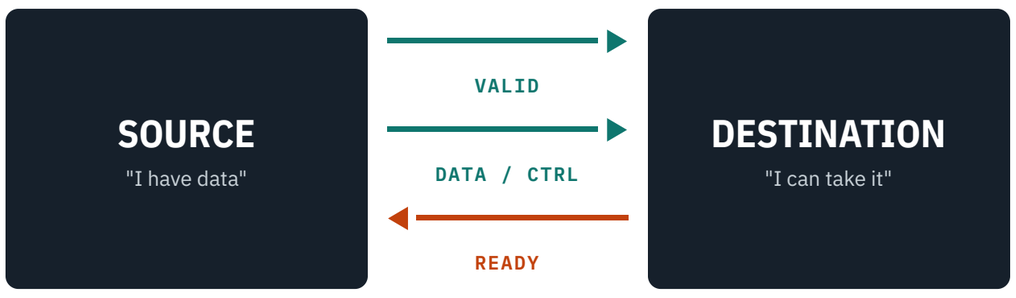
DV INTERVIEW · GLANCE SHEET 4

The VALID–READY Handshake, In Depth

One mechanism, used identically on all five channels. Get this right and most AXI questions answer themselves.

1 HIGH
⇄ handshake edge
✗ protocol violation

1 The idea in one picture



Transfer = VALID && READY at a rising ACLK edge. Nothing else moves data.

Channel	Source (VALID)	Destination (READY)
AW, W, AR	Master	Slave
B, R	Slave	Master

2 The five rules

#	Rule	Why
R1	Source **must not wait for READY** before asserting VALID.	If both sides wait for each other → deadlock.
R2	Once VALID is HIGH it **stays HIGH until the handshake**.	Destination may sample any later cycle; data can't vanish.
R3	While VALID is HIGH and not yet accepted, **payload stays stable**.	Destination must get exactly what was offered.
R4	Destination **may wait for VALID** before asserting READY.	Lets slaves be simple and save power.
R5	READY **may be asserted early** and **may be de-asserted** before VALID arrives.	READY is only a "can accept now" hint until VALID shows up.

Asymmetry to remember: VALID is a *commitment* (can't take it back). READY is an *offer* (can withdraw it while VALID is low).

3 The three legal cases

CASE A · VALID BEFORE READY

ACLK	T1	T2	T3	T4	T5	T6
VALID	0	1	1	⇄	0	0
READY	0	0	0	⇄	0	0
DATA	–	D	D	D	–	–

Source waits with D held stable (R2, R3). Typical when the slave is busy.

CASE B · READY BEFORE VALID

ACLK	T1	T2	T3	T4	T5	T6
VALID	0	0	0	⇄	0	0
READY	0	1	1	⇄	0	0
DATA	–	–	–	D	–	–

Destination is idle and waiting. Transfer happens the first cycle VALID rises — zero wait.

CASE C · BOTH IN THE SAME CYCLE

ACLK	T1	T2	T3	T4	T5	T6
VALID	0	0	⇄	0	0	0
READY	0	0	⇄	0	0	0
DATA	–	–	D	–	–	–

Both assert together → transfer in that same cycle. One beat per cycle is the max rate.

Back-to-back streaming: keep VALID and READY both HIGH and change DATA every cycle → one transfer per clock (100% throughput). Every cycle with VALID && !READY or READY && !VALID is a bubble.

✗ Illegal cases — the ones interviewers draw on the board

VIOLATION 1 · VALID DROPS EARLY (BREAKS R2)

ACLK	T1	T2	T3	T4	T5	T6
VALID	0	1	0 ✗	1	0	0
READY	0	0	0	0	0	0

Source "took back" its offer at T3 without a handshake.

VIOLATION 2 · DATA CHANGES WHILE STALLED (BREAKS R3)

ACLK	T1	T2	T3	T4	T5	T6
VALID	0	1	1	⇄	0	0
READY	0	0	0	⇄	0	0
DATA	–	D0	D1 ✗	D1	–	–

D0 was offered but D1 got accepted — D0 silently lost.

VIOLATION 3 · SOURCE WAITS FOR READY (BREAKS R1)

ACLK	T1	T2	T3	T4	T5	T6
VALID	0	0	0	0	0	0 …
READY	0	0	0	0	0	0 …

Source waits for READY, slave waits for VALID (allowed by R4) → **deadlock forever**.

Not a violation: READY going HIGH then LOW while VALID is LOW (R5). Also legal: VALID HIGH for many cycles with READY LOW — that's just back-pressure.

4 Cross-channel dependencies

READ

`ARVALID` + `ARREADY` must both happen before `RVALID`

ARREADY itself may be asserted before or after ARVALID.

WRITE (AXI4)

`AW handshake` + `W handshake with WLAST` must both happen before `BVALID`

- Slave may wait for AWVALID, WVALID or both before raising AWREADY.
- Slave may wait for AWVALID, WVALID or both before raising WREADY.
- Master must **not** wait for AWREADY before WVALID, nor for WREADY before AWVALID.
- AXI3 was looser: BVALID only had to follow WLAST, not AW.

Mental model: a slave can always say "I'll wait for you". A master can never say "you go first".

5 Timing & implementation notes

- Everything is sampled on the **rising edge** of ACLK.
- **No combinational path** from inputs to outputs on an interface — e.g. READY must not be a pure function of VALID through logic in the same cycle at the interface boundary.
- READY travels **backward**; long READY paths hurt timing → insert a **register slice**.
- A full-throughput register slice needs a **2-entry skid buffer**: when READY is registered, one extra beat may arrive after downstream stalls, so it must be stored.
- During reset, all VALIDs LOW; first VALID only after ARESETn HIGH.
- Good practice: masters keep `BREADY` / `RREADY` HIGH unless they truly can't accept.

Throughput math: efficiency = handshakes ÷ cycles. 4 beats over 6 cycles = 67%; 2 bubbles came from back-pressure or idle VALID.

6 Handshake assertions (SVA)

R2 — VALID HELD UNTIL READY

```
property p_valid_hold;
  @(posedge ACLK) disable iff (!ARESETn)
  AWVALID && !AWREADY |=> AWVALID;
endproperty
```

R3 — PAYLOAD STABLE WHILE STALLED

```
property p_payload_stable;
  @(posedge ACLK) disable iff (!ARESETn)
  AWVALID && !AWREADY |=>
    $stable({AWADDR, AWLEN, AWSIZE, AWBURST, AWID});
endproperty
```

RESET & X CHECKS

```
a_rst_low : assert property (@(posedge ACLK) !ARESETn |-> !AWVALID);
a_no_x    : assert property (@(posedge ACLK) disable iff (!ARESETn)
                             !$isunknown(AWVALID));
```

Copy the same set for W, AR (master side) and B, R (slave side). Add a liveness/timeout check: VALID must eventually see READY within N cycles.

? Handshake interview Q&A

Question	Crisp answer
When exactly does a transfer happen?	Rising edge where VALID and READY are both HIGH.
Can VALID depend on READY?	No — that's the deadlock rule (R1).
Can READY depend on VALID?	Yes — destination may wait for VALID.
Can READY drop without a transfer?	Yes, if VALID was LOW.
Can VALID drop without a transfer?	No — protocol violation.
Max throughput of one channel?	One transfer per clock (VALID & READY both held HIGH).
Why a skid buffer in a register slice?	Registered READY lags by a cycle; the in-flight beat needs a slot.
How do you test the handshake in a TB?	Randomize VALID/READY delays incl. 0; cover cases A/B/C; SVA for hold/stable/reset.
Bug: slave waits for WVALID before AWREADY, master waits for AWREADY before WVALID?	Deadlock; the master is the one breaking the rule.

DV INTERVIEW · GLANCE SHEET 5

Bursts: Length, Size & Type

Three fields decide every address in a burst. Master them and you can compute any beat, lane and strobe on a whiteboard.

```
beats = AxLEN + 1
bytes/beat = 2^AxSIZE
max bytes = (LEN+1) × 2^SIZE
```

AXLEN · HOW MANY

Burst length

Number of data beats − 1

Type	AXI3	AXI4
INCR	1–16	**1–256**
WRAP	2, 4, 8, 16	2, 4, 8, 16
FIXED	1–16	1–16
Field width	4 bits	8 bits

`AxLEN = 0` means **1** beat. `AxLEN = 8'hFF` means 256 beats.

AXSIZE · HOW WIDE

Burst size

Bytes in each beat = 2^SIZE

SIZE	Bytes	SIZE	Bytes
000	1	100	16
001	2	101	32
010	4	110	64
011	8	111	128

2^SIZE must be **≤ data bus width**. Smaller than the bus = **narrow transfer**.

AXBURST · WHICH PATTERN

Burst type

How the address moves beat to beat

Code	Type	Address	Use
00	**FIXED**	Same each beat	FIFO, data port
01	**INCR**	+2^SIZE each beat	Normal memory
10	**WRAP**	+2^SIZE, wraps	Cache line fill
11	—	Reserved	Error

INCR is the **default** and the most common. AXI-Lite has no bursts at all.

ƒ Address & byte-lane formulas (works for every burst)

DEFINITIONS

```
N_bytes   = 2 ** AxSIZE              // bytes per beat
Beats     = AxLEN + 1
Aligned   = (Start / N_bytes) * N_bytes   // integer divide
Bus_bytes = DATA_WIDTH / 8
```

ADDRESS OF BEAT N (N = 1 … BEATS)

```
FIXED : Addr[n] = Start
INCR  : Addr[1] = Start
        Addr[n] = Aligned + (n-1) * N_bytes      // n ≥ 2
WRAP  : Lower   = (Start / (N_bytes*Beats)) * (N_bytes*Beats)
        Upper   = Lower + N_bytes*Beats
        Addr[n] = INCR formula, but if it hits Upper → Lower
```

BYTE LANES OF BEAT N (FOR WSTRB)

```
Low_lane  = Addr[n] % Bus_bytes
High_lane = (Aligned_n % Bus_bytes) + N_bytes - 1
// Aligned_n = Addr[n] rounded down to N_bytes
// first beat may start mid-way (unaligned)
```

TOTAL BYTES ACTUALLY MOVED

```
Total = Beats * N_bytes - (Start % N_bytes)
// the subtracted part is the unaligned head
```

Only the first beat can be unaligned. From beat 2 onward INCR addresses are always multiples of N_bytes.

I INCR burst in detail — 4 worked examples

EX 1 · ALIGNED, FULL WIDTH · 32-BIT BUS · START 0X1000 · LEN 7 · SIZE 010

`0x1000` `0x1004` `0x1008` `0x100C` `0x1010` `0x1014` `0x1018` `0x101C`

8 beats × 4 B = **32 bytes**. Every WSTRB = `1111`. Last byte 0x101F.

EX 2 · NARROW · 64-BIT BUS (8 LANES) · START 0X04 · LEN 3 · SIZE 010 (4 B)

0x04	**7**	**6**	**5**	**4**	3	2	1	0	`WSTRB 0xF0`
0x08	7	6	5	4	**3**	**2**	**1**	**0**	`WSTRB 0x0F`
0x0C	**7**	**6**	**5**	**4**	3	2	1	0	`WSTRB 0xF0`
0x10	7	6	5	4	**3**	**2**	**1**	**0**	`WSTRB 0x0F`

The 4-byte window **ping-pongs** between upper and lower halves of the bus.

EX 3 · UNALIGNED · 32-BIT BUS · START 0X07 · LEN 2 · SIZE 010

0x07	**3**	2	1	0	`1000`	1 byte
0x08	**3**	**2**	**1**	**0**	`1111`	4 bytes
0x0C	**3**	**2**	**1**	**0**	`1111`	4 bytes

Aligned = 0x04, so beat 2 = 0x04 + 4 = **0x08** (not 0x0B). Total = 3×4 − 3 = **9 bytes**.

EX 4 · THE 4 KB BOUNDARY CHECK

`0xFF0` LEN 3, SIZE 010 → 16 B → last byte `0xFFF` ✓ legal, ends exactly at boundary

`0xFF8` LEN 3, SIZE 010 → 16 B → last byte `0x1007` × crosses 0x1000

Fix: split into two bursts — 0xFF8 (LEN 1, 8 B) and 0x1000 (LEN 1, 8 B). Interconnects / DMA engines do this automatically.

Check: address bits [MSB:12] of the first byte and last byte must be equal.

F FIXED burst

32-BIT BUS · START 0X20 · LEN 3 · SIZE 010

`0x20` `0x20` `0x20` `0x20`

- Same address, same byte lanes on every beat.
- Used to push/pop a **FIFO** or a peripheral data register.
- Max 16 beats in AXI3 and AXI4.
- Rarely supported by memory slaves — cover it only if the DUT claims support.

W WRAP burst

32-BIT BUS · START 0X14 · LEN 7 (8 BEATS) · SIZE 010

Container = 8 × 4 = **32 B** → Lower = INT(0x14 / 32) × 32 = **0x00**, Upper = **0x20**.

`0x14` `0x18` `0x1C` → hits 0x20, wrap → `0x00` `0x04` `0x08` `0x0C` `0x10`

- Length must be 2, 4, 8 or 16; start must be **aligned to SIZE**.
- Total container is always a power of two, so a WRAP never crosses 4 KB.
- Why: CPU misses on word 0x14 → that word returns **first** ("critical word first"), rest of the cache line follows.
- If start is already at Lower, a WRAP looks exactly like an INCR.

✓ Burst constraints & coverage (SV)

```
class axi_txn extends uvm_sequence_item;
  rand bit [31:0] addr;
  rand bit [7:0]  len;
  rand bit [2:0]  size;
  rand bit [1:0]  burst;

  constraint c_type  { burst != 2'b11; }
  constraint c_size  { (1 << size) <= BUS_BYTES; }
  constraint c_len   {
    burst == WRAP  -> len inside {1, 3, 7, 15};
    burst == FIXED -> len < 16; }
  constraint c_wrap  { burst == WRAP -> addr % (1 << size) == 0; }
  constraint c_4k    { burst == INCR ->
    (addr % 4096) + (len + 1) * (1 << size) <= 4096; }
endclass
```

c_4k is safe (slightly conservative for unaligned starts). Use `solve burst before len` for better distribution.

COVERAGE

- **cross burst × len_bins × size** (len bins: 1, 2–15, 16, 17–255, 256).
- aligned vs unaligned start × narrow vs full width.
- INCR ending exactly at a 4 KB boundary; WRAP starting at every offset in the container.

? Burst interview Q&A

Question	Crisp answer
LEN = 15, SIZE = 3 — bytes?	16 × 8 = 128 bytes.
Max bytes in one AXI4 INCR on a 128-bit bus?	256 × 16 = 4096 — but 4 KB rule means it must start at a 4 KB-aligned address.
Why the 4 KB limit?	Smallest slave region / page; a burst must stay in one slave.
Why only 16 beats for WRAP?	It's for cache lines, which are small; power-of-2 container needed.
Can a burst stop early?	No — all LEN+1 beats must transfer; use WSTRB = 0 to skip writes.
Next address after unaligned 0x07, SIZE 4?	0x08 (aligned base 0x04 + 4).
Can SIZE be larger than the bus?	No — illegal.
Can WRAP start unaligned?	No — start must be a multiple of 2^SIZE.
WRAP 4 beats, 8 B, start 0x18?	Container 32 B → 0x18, 0x00, 0x08, 0x10.

DV INTERVIEW · GLANCE SHEET 6

The WRAP Burst, Step by Step

An INCR burst that lives inside a fixed, aligned window and loops back to the window's start when it reaches the end.

```
Window = (LEN+1) × 2^SIZE
Lower = INT(Start / Window) × Window
Upper = Lower + Window → jump to Lower
```

1 Why WRAP exists: critical word first

- A CPU misses in its cache on one word, e.g. `0x14`, and has to fetch the whole cache line `0x00–0x1F`.
- With INCR it would have to start at 0x00 and wait for 0x14 to arrive 6th.
- With WRAP it starts at **0x14** — the word the CPU is stalled on comes back **first** — then the burst wraps round to fill the rest of the line.
- Result: lower miss latency with a single burst, and the line is still fetched completely.

One-line answer: WRAP fetches a whole aligned block but in an order that starts at the address you actually need.

2 Rules of a legal WRAP

#	Rule	Reason
1	Length = **2, 4, 8 or 16** beats (AxLEN = 1, 3, 7, 15).	Window must be a power of two to be self-aligned.
2	Start address **aligned to SIZE** (Start % 2^SIZE = 0).	Every beat must fit exactly in the window.
3	2^SIZE ≤ data bus width.	Same as any burst.
4	AxBURST = `2'b10`.	—

Never crosses 4 KB. Max window = 16 × 128 B = 2 KB, and it's aligned to its own size, so it always sits inside one 4 KB page — no 4 KB check needed.

3 The algorithm — 4 steps you can do on a whiteboard

Step 1 · Window size
`W = (LEN+1) × 2^SIZE`
e.g. 8 beats × 4 B = 32 B

Step 2 · Lower boundary
`L = INT(Start / W) × W`
(= Start with low bits cleared)

Step 3 · Upper boundary
`U = L + W`
first address *outside* the window

Step 4 · Walk
add 2^SIZE each beat; when address = U, set it to L. Stop after LEN+1 beats.

REFERENCE MODEL (SYSTEMVERILOG)

```
function automatic void wrap_addrs(
    input  bit [31:0] start, input bit [7:0] len,
    input  bit [2:0]  size,  ref bit [31:0] q[$]);
  int unsigned nb    = 1 << size;
  int unsigned win   = nb * (len + 1);
  bit [31:0]   lower = (start / win) * win;
  bit [31:0]   upper = lower + win;
  bit [31:0]   a     = start;
  repeat (len + 1) begin
    q.push_back(a);
    a += nb;
    if (a == upper) a = lower;
  end
endfunction
```

BIT-TRICK VERSION (WINDOW IS A POWER OF 2)

```
mask  = W - 1;              // e.g. 32 B → 0x1F
lower = start & ~mask;
next  = lower | ((addr + nb) & mask);
```

Only the low log2(W) bits move; upper bits stay fixed. That's exactly how RTL address generators implement WRAP — no comparator needed.

Shortcut: the number of beats before the wrap = (U − Start) / 2^SIZE. Beats after the wrap = the rest.

Special case: if Start = L, the burst never wraps and is identical to INCR.

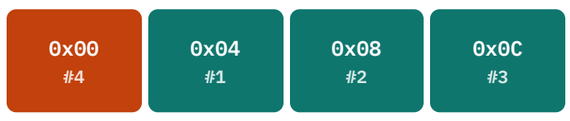
4 One window, every starting point — 4 beats × 4 B (window 16 B, 0x00–0x0F)

START 0X00 (= INCR)



No wrap.

START 0X04



04 → 08 → 0C → **00**

START 0X08



08 → 0C → **00** → **04**

START 0X0C



0C → **00** → **04** → **08**

Teal = before the wrap, orange = after. The **same 16 bytes** are always covered — only the order changes, rotating to start at the requested word.

5 More worked examples

Case	LEN / beats	SIZE	Start	Window	L → U	Address sequence
A · 2-beat	1 / 2	8 B	`0x38`	16 B	0x30 → 0x40	`0x38, 0x30`
B · cache line	7 / 8	4 B	`0x14`	32 B	0x00 → 0x20	`0x14, 0x18, 0x1C,` **wrap** `0x00, 0x04, 0x08, 0x0C, 0x10`
C · late start	7 / 8	8 B	`0x1F8`	64 B	0x1C0 → 0x200	`0x1F8,` **wrap** `0x1C0, 0x1C8, 0x1D0, 0x1D8, 0x1E0, 0x1E8, 0x1F0`
D · 16-beat	15 / 16	4 B	`0x2C`	64 B	0x00 → 0x40	`0x2C, 0x30, 0x34, 0x38, 0x3C,` **wrap** `0x00, 0x04 … 0x24, 0x28` (11 more)
E · high address	3 / 4	16 B	`0x8020`	64 B	0x8000 → 0x8040	`0x8020, 0x8030,` **wrap** `0x8000, 0x8010`

NARROW WRAP · 64-BIT BUS · SIZE 4 B · 4 BEATS · START 0X108

Window 16 B → L = 0x100, U = 0x110 → `0x108, 0x10C, 0x100, 0x104`



WRAP VS INCR, SAME REQUEST

Start 0x14 · 8 × 4 B	Sequence	Bytes covered
INCR	`0x14 … 0x30`	0x14–0x33 (spills into next line)
WRAP	`0x14 … 0x1C, 0x00 … 0x10`	0x00–0x1F (exactly one line)

Same LEN/SIZE, same first beat — but WRAP stays inside the aligned 32 B line while INCR runs past it.

✗ Illegal WRAPs

Request	Why illegal
LEN = 5 (6 beats)	6 is not 2/4/8/16
LEN = 0 (1 beat)	Minimum WRAP length is 2
Start 0x06, SIZE 4 B	Start not aligned to 4
LEN = 31 in AXI4	WRAP max is 16 (only INCR grew to 256)
SIZE 8 B on a 32-bit bus	Beat wider than the bus

✓ Verifying WRAP

```
constraint c_wrap {
  burst == WRAP -> len inside {1, 3, 7, 15};
  burst == WRAP -> addr % (1 << size) == 0;
}
```

- Scoreboard uses `wrap_addrs()` to know which byte each beat hits.
- Cover: **every start offset in the window** (start = L, mid, last slot) × len {2,4,8,16} × size.
- Cover narrow WRAP and WRAP at max size.
- Negative tests (if DUT must flag them): unaligned start, bad LEN → expect SLVERR or assertion.
- Assertion: every WRAP beat address stays in [L, U).

? WRAP interview Q&A

Question	Crisp answer
What is a WRAP burst?	INCR that loops back to the start of an aligned window of size (LEN+1)×2^SIZE.
Main use?	Cache line fill — critical word first.
Allowed lengths?	2, 4, 8, 16 beats.
Why must the length be a power of 2?	So the window is self-aligned and address bits wrap naturally.
Can WRAP cross 4 KB?	No — window ≤ 2 KB and aligned to its size.
Can WRAP start unaligned?	No — must be aligned to 2^SIZE.
WRAP 4 × 4 B from 0x2C?	Window 0x20–0x2F → 0x2C, 0x20, 0x24, 0x28.
WRAP 8 × 2 B from 0x0A?	Window 0x00–0x0F → 0x0A, 0x0C, 0x0E, 0x00, 0x02, 0x04, 0x06, 0x08.
When is WRAP identical to INCR?	When the start address is the lower boundary.
How does RTL generate the next address?	Increment only the low log2(window) bits; upper bits fixed.
Does WRAP change WSTRB rules?	No — lanes follow each beat's address, as for INCR.

DV INTERVIEW · GLANCE SHEET 7

Out-of-Order Transactions & IDs

How AXI lets fast responses overtake slow ones — and the single rule that keeps it safe.

Same ID → in issue order
Different ID → any order

1 What & why

- **Out-of-order (OoO)**: responses come back in a different order than the requests were issued.
- Possible because AXI has **multiple outstanding transactions** and tags each with an **ID**.
- Why it matters: slaves have very different latencies — an SRAM answers in 2 cycles, DDR in 50+. In-order would make every fast access wait behind the slow one (**head-of-line blocking**).
- DDR controllers also reorder internally (open-row hits first), so they *need* OoO to be efficient.

One-liner: IDs let the system return each response as soon as it's ready, while preserving order only where the master needs it.

2 The ordering rules

#	Rule
1	**Same ID, same direction** → responses must return in the order the addresses were issued.
2	**Different IDs** → no ordering requirement; may complete in any order.
3	Read data **may interleave** between different RIDs; beats **within one burst stay in order**.
4	Write responses (B) may return OoO for different BIDs.
5	**AXI4 write data** must follow AW order — no interleaving (AXI3 allowed it via WID).
6	**No ordering between reads and writes**, even with the same ID. Need it? Wait for the response first.

A master that always uses **one ID** gets fully in-order behaviour for free. Using several IDs is an opt-in to OoO.

3 Same request, two ID choices — A to slow DDR (4 beats), B to fast SRAM (2 beats)

DIFFERENT IDS (A = ID0, B = ID1) → OUT-OF-ORDER ALLOWED



B finishes at cycle **5**, A at **10**. B overtook A — legal because the IDs differ.

SAME ID (BOTH ID0) → MUST STAY IN ORDER



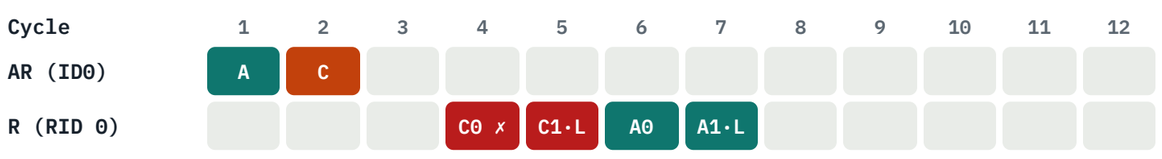
B's data was ready at cycle 4 but is **blocked behind A** until cycle 11 → head-of-line blocking.

READ DATA INTERLEAVING (DIFFERENT RIDS, BEAT LEVEL)



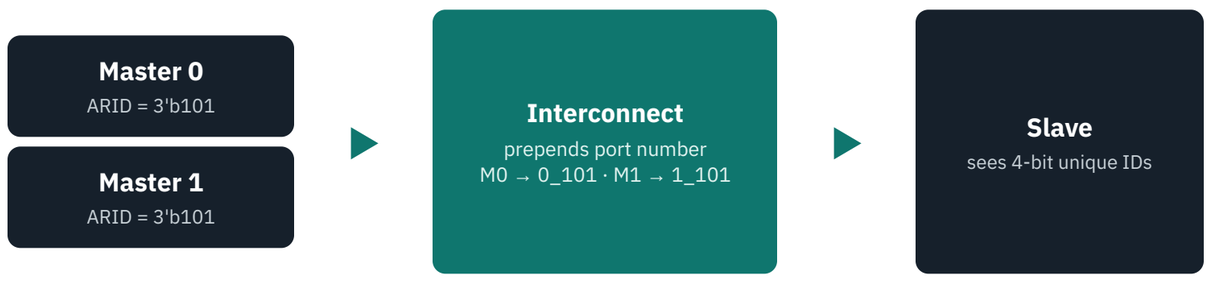
Legal: beats of A and B mixed. Illegal: A1 before A0 (order *within* a burst is fixed).

ILLEGAL: SAME ID RETURNED OUT OF ORDER



Master would think C's data belongs to A — data corruption. Scoreboards must flag this.

4 What the interconnect does with IDs



- **ID widening:** slave-side ID = master-side ID + log2(#masters) bits. Two masters can reuse the same ID without clashing.
- **Response routing:** the extra bits on RID/BID tell the interconnect which master to send the response to; it strips them on the way back.
- **Same ID to two different slaves:** the slaves can't coordinate, so the interconnect must **stall** the 2nd request until the 1st completes (or reorder responses itself). This is the "single-slave per ID" rule many fabrics use.

Interview favourite: "Master sends ID0 to slave X, then ID0 to slave Y. Y is faster — what happens?" → interconnect blocks or buffers so X's response still reaches the master first.

5 Write side & capabilities

Item	AXI3	AXI4
B responses OoO (diff BID)	Yes	Yes
W data interleaving	Yes (WID)	**No**
W data order	By WID	Same order as AW
R data interleaving	Yes	Yes

PARAMETERS TO KNOW FOR ANY DUT

- **ID width** — how many distinct streams a master can use.
- **Read / write acceptance capability** — max outstanding transactions a slave accepts.
- **Issuing capability** — max outstanding a master will issue.
- **Read data reordering depth** — how many different IDs a slave may reorder between.
- **Read interleaving depth** — how many bursts can be active on R at once (1 = no interleave).

Reordering depth 1 = slave is **in-order**. Always legal, just slower.

✓ Verifying OoO — per-ID scoreboard

```
class axi_rd_sb extends uvm_scoreboard;
  // one FIFO of expected transactions per ID
  axi_txn exp_q[bit [ID_W-1:0]][$];

  function void write_ar(axi_txn t);   // from AR monitor
    exp_q[t.id].push_back(t);
  endfunction

  function void write_r(axi_txn t);    // full burst from R monitor
    axi_txn e;
    if (!exp_q.exists(t.id) || exp_q[t.id].size() == 0)
      `uvm_error("SB", $sformatf("RID %0h not outstanding", t.id))
    else begin
      e = exp_q[t.id].pop_front();     // oldest of SAME ID
      if (!t.data_match(e)) `uvm_error("SB", "data mismatch")
    end
  endfunction
endclass
```

Popping the **front of the same-ID queue** enforces rule 1; using a queue per ID permits rule 2 automatically.

CHECKS & COVERAGE

- RID/BID always matches an outstanding ID; no response for an ID with an empty queue.
- Monitor collects R beats **per RID** so interleaved bursts rebuild correctly.
- Cover: OoO actually happened (completion order ≠ issue order), same-ID back-to-back, max outstanding, interleaved R, all ID values.
- Slave VIP knob: random response reordering + random interleaving.

? Out-of-order interview Q&A

Question	Crisp answer
What enables OoO in AXI?	Multiple outstanding transactions + ID tags.
What is the ordering rule?	Same ID in order; different IDs any order.
Can beats inside one burst reorder?	No — only whole transactions with different IDs.
Can read data interleave?	Yes, across different RIDs.
Can write data interleave in AXI4?	No — WID removed; W follows AW order.
Is a write ordered with a later read of the same ID?	No — wait for BRESP before issuing the read.
How does the response find its master?	Interconnect-added ID bits route it back.
Same ID to two slaves — risk?	Order violation; interconnect must stall or reorder.
Benefit of OoO?	Removes head-of-line blocking; better throughput and latency.
Cost of OoO?	ID tracking and reorder buffers in master/interconnect.
How do you test it?	Per-ID queue scoreboard, reordering slave VIP, coverage on completion order.

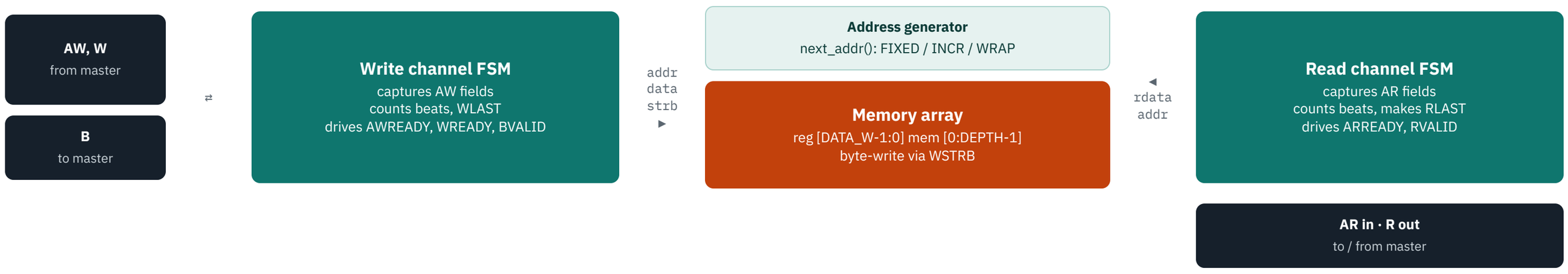
DV INTERVIEW · GLANCE SHEET 8 · PROJECT PART 1

Building an AXI Slave RAM in Verilog

A memory that speaks AXI: two independent state machines, an address generator, and byte-enabled writes.

`Write FSM: IDLE → DATA → RESP`
`Read FSM: IDLE → DATA`
`mem index = addr >> log2(DATA_W/8)`

1 Architecture

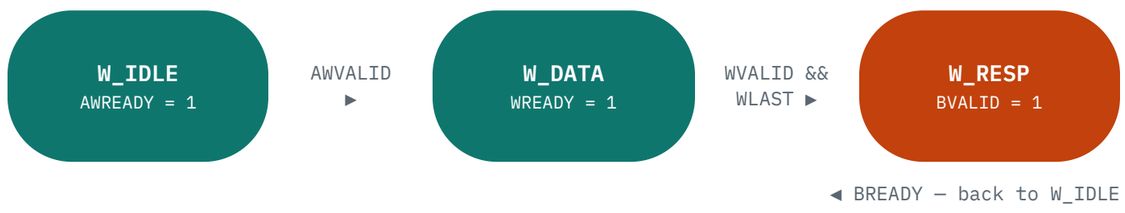


Read and write FSMs are independent → a read and a write can be in progress at the same time, just as AXI intends.

Address generator is shared logic — the same function is used by both FSMs, so burst rules are coded once.

Parameters: `ADDR_W`, `DATA_W`, `ID_W`, `DEPTH`. STRB width = DATA_W/8.

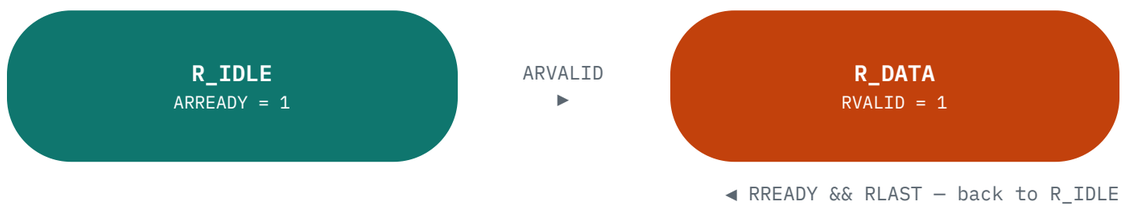
2 Write FSM



State	Does	Leaves when
W_IDLE	AWREADY HIGH; on handshake latch AWID, AWADDR, AWLEN, AWSIZE, AWBURST	AWVALID && AWREADY
W_DATA	WREADY HIGH; each WVALID beat writes memory with WSTRB, advances address	WVALID && WREADY && WLAST
W_RESP	BVALID HIGH with BID = latched AWID, BRESP = OKAY / SLVERR	BVALID && BREADY

This slave **waits for AW before accepting W**. That's legal (a slave may wait for AWVALID before WREADY) and keeps the design simple, but your **master/TB must not wait for WREADY before AWVALID**.

3 Read FSM



State	Does	Leaves when
R_IDLE	ARREADY HIGH; latch ARID, ARADDR, ARLEN, ARSIZE, ARBURST; clear beat counter	ARVALID && ARREADY
R_DATA	RVALID HIGH; RDATA = mem[addr]; RLAST = (count == ARLEN); on each handshake advance address & count	RVALID && RREADY && RLAST

RDATA stays stable while RREADY is LOW because the address only moves on a handshake — the handshake rule is met for free.

With a real **synchronous** block RAM, data appears a cycle after the address. Then you need a pre-fetch stage or a small output register/skid buffer so RVALID and RDATA line up.

4 Key Verilog pieces

WRITE FSM (CORE)

```
localparam W_IDLE = 2'd0, W_DATA = 2'd1, W_RESP = 2'd2;

always @(posedge ACLK or negedge ARESETn)
  if (!ARESETn) wstate <= W_IDLE;
  else case (wstate)
    W_IDLE: if (AWVALID) begin            // AWREADY = 1 here
              aw_id   <= AWID;    w_addr  <= AWADDR;
              aw_len  <= AWLEN;   aw_size <= AWSIZE;
              aw_burst<= AWBURST; wstate  <= W_DATA;
            end
    W_DATA: if (WVALID) begin             // WREADY = 1 here
              w_addr <= next_addr(w_addr, aw_len, aw_size, aw_burst);
              if (WLAST) wstate <= W_RESP;
            end
    W_RESP: if (BREADY) wstate <= W_IDLE; // BVALID = 1 here
  endcase

assign AWREADY = (wstate == W_IDLE);
assign WREADY  = (wstate == W_DATA);
assign BVALID  = (wstate == W_RESP);
assign BID     = aw_id;
assign BRESP   = 2'b00;                   // OKAY
```

BYTE-ENABLED MEMORY WRITE

```
integer i;
always @(posedge ACLK)
  if (WVALID && WREADY)
    for (i = 0; i < DATA_W/8; i = i + 1)
      if (WSTRB[i])
        mem[w_addr >> ADDR_LSB][8*i +: 8] <= WDATA[8*i +: 8];
// ADDR_LSB = log2(DATA_W/8), e.g. 2 for 32-bit
```

READ FSM (CORE)

```
localparam R_IDLE = 1'b0, R_DATA = 1'b1;

always @(posedge ACLK or negedge ARESETn)
  if (!ARESETn) rstate <= R_IDLE;
  else case (rstate)
    R_IDLE: if (ARVALID) begin            // ARREADY = 1 here
              ar_id <= ARID;   r_addr  <= ARADDR;
              ar_len<= ARLEN;  ar_size <= ARSIZE;
              ar_burst <= ARBURST; r_cnt <= 0;
              rstate <= R_DATA;
            end
    R_DATA: if (RREADY) begin             // RVALID = 1 here
              r_addr <= next_addr(r_addr, ar_len, ar_size, ar_burst);
              r_cnt  <= r_cnt + 1;
              if (RLAST) rstate <= R_IDLE;
            end
  endcase

assign ARREADY = (rstate == R_IDLE);
assign RVALID  = (rstate == R_DATA);
assign RDATA   = mem[r_addr >> ADDR_LSB];
assign RLAST   = (r_cnt == ar_len);
assign RID     = ar_id;
assign RRESP   = 2'b00;
```

NEXT-ADDRESS FUNCTION (ALL BURST TYPES)

```
function [ADDR_W-1:0] next_addr(input [ADDR_W-1:0] a,
    input [7:0] len, input [2:0] size, input [1:0] burst);
  reg [ADDR_W-1:0] nb, win, inc;
  begin
    nb  = 1 << size;
    inc = (a & ~(nb - 1)) + nb;          // align, then step
    win = nb * (len + 1);
    case (burst)
      2'b00:   next_addr = a;                              // FIXED
      2'b10:   next_addr = (a & ~(win-1)) | (inc & (win-1)); // WRAP
      default: next_addr = inc;                            // INCR
    endcase
  end
endfunction
```

5 Design choices

Choice	Effect
One write + one read in flight	Simple; no ID queues; always in-order
Separate R / W FSMs	Reads and writes run concurrently
Wait for AW before W	Legal; no W buffering needed
Align address after beat 1	Handles unaligned starts correctly
Word-indexed memory + WSTRB	Narrow and unaligned writes work
Out-of-range address	Return SLVERR (still finish all beats)
BURST = 2'b11	Treat as error → SLVERR

Upgrade path: add AW/AR FIFOs for multiple outstanding, then per-ID tracking to allow out-of-order.

! Classic bugs in AXI slaves

- WREADY HIGH before AW is latched → data written to the wrong address.
- RLAST off by one (counter compared to LEN+1 instead of LEN).
- BVALID dropped before BREADY → response lost.
- Beat-2 address = start + size instead of **aligned** start + size.
- Ignoring WSTRB → narrow writes clobber neighbour bytes.
- VALIDs not forced LOW in reset.
- Ending the burst on count but ignoring WLAST mismatch (should flag).
- Using AW fields directly instead of latched copies after handshake.
- RDATA changing while RVALID HIGH and RREADY LOW.

? Test plan & Q&A

DIRECTED TESTS FIRST

- Single write → read back.
- INCR / WRAP / FIXED burst write, then read back.
- Partial WSTRB, narrow and unaligned writes.
- BREADY / RREADY held LOW for N cycles.
- Read and write at the same time.
- Reset in the middle of a burst.

Question	Answer
Why two FSMs?	R and W channels are independent.
Why latch AW fields?	Master may change AW after the handshake.
How is WSTRB used?	Per-byte write enable on the memory word.
Is this slave in-order?	Yes — one transaction per direction at a time.

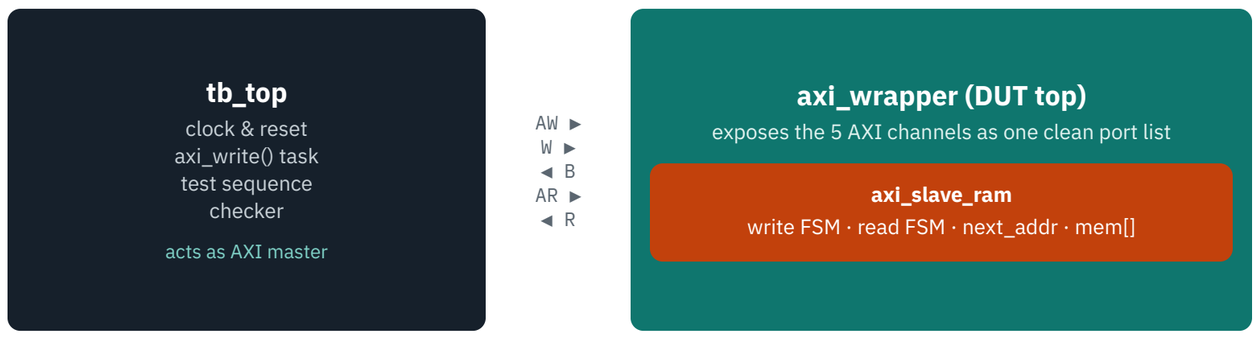
DV INTERVIEW · GLANCE SHEET 9 · PROJECT PART 2

Writing to the RAM from a Testbench

The testbench plays AXI master: it drives AW and W, waits for the handshakes, collects B, then checks memory.

```
drive with <= right after posedge
sample READY at the next posedge
AW ‖ W in fork…join, then B
```

1 Testbench structure & the AXI wrapper



- **Wrapper** = a thin top module that groups the AXI ports and instantiates the RAM slave. The TB only ever talks to the wrapper, so the core can change without touching the TB.
- Same idea in UVM: the wrapper's ports bind to an **AXI interface**, and drivers/monitors use that interface.

2 Clock, reset & idle values

```
initial ACLK = 0;
always #5 ACLK = ~ACLK;          // 100 MHz

initial begin
  ARESETn = 0;
  AWVALID = 0; WVALID = 0; WLAST = 0;
  BREADY  = 0; ARVALID = 0; RREADY = 0;
  repeat (5) @(posedge ACLK);
  ARESETn <= 1;                  // de-assert on an edge
  @(posedge ACLK);
  run_write_tests();
  $finish;
end
```

All master VALIDs must be LOW during reset and stay LOW until after ARESETn goes HIGH — the TB is the master, so this is **your** responsibility.

3 The reusable `axi_write` task (SystemVerilog)

```
logic [DATA_W-1:0] wdata_q[$];   // filled by the test before calling
logic [STRB_W-1:0] wstrb_q[$];

task automatic axi_write(input [ID_W-1:0] id, input [ADDR_W-1:0] addr,
                         input [7:0] len, input [2:0] size, input [1:0] burst);
  fork
    begin : aw_ch                                  // ---- address channel
      AWID <= id;  AWADDR <= addr;  AWLEN <= len;
      AWSIZE <= size;  AWBURST <= burst;  AWVALID <= 1;
      do @(posedge ACLK); while (!AWREADY);      // handshake edge
      AWVALID <= 0;
    end
    begin : w_ch                                   // ---- data channel
      for (int i = 0; i <= len; i++) begin
        WDATA <= wdata_q[i];  WSTRB <= wstrb_q[i];
        WLAST <= (i == len);  WVALID <= 1;
        do @(posedge ACLK); while (!WREADY);     // one beat accepted
      end
      WVALID <= 0;  WLAST <= 0;
    end
  join

  BREADY <= 1;                                     // ---- response channel
  do @(posedge ACLK); while (!BVALID);
  BREADY <= 0;
  if (BID !== id)      $error("BID %0h != AWID %0h", BID, id);
  if (BRESP !== 2'b00) $error("BRESP = %b at addr %h", BRESP, addr);
endtask
```

fork…join runs AW and W at the same time. The TB raises WVALID without waiting for AWREADY — exactly what the protocol requires of a master (no "wait for READY before VALID").

do @(posedge) while(!READY) — after the edge, READY still shows the value the DUT had *at* that edge (the DUT's registers update later via NBA), so this samples the handshake correctly.

Non-blocking `<=` for every drive avoids races with the DUT on the same clock edge.

Payload is held automatically: nothing changes until the loop sees READY, so VALID-stable / data-stable rules hold.

Add a **timeout** around each wait (e.g. a parallel `repeat(1000) @(posedge ACLK)` + `$fatal`) so a stuck READY doesn't hang the sim forever.

4 What a 2-beat write looks like (with the Part-1 slave)

ACLK	T1	T2	T3	T4	T5	T6	T7	T8
AWVALID	⇄	0	0	0	0	0	0	0
AWREADY	⇄	0	0	0	1	1	1	1
WVALID	1	⇄	⇄	0	0	0	0	0
WREADY	0	⇄	⇄	0	0	0	0	0
WDATA	D0	D0	D1	–	–	–	–	–
WLAST	0	0	1	0	0	0	0	0
BVALID	0	0	0	⇄	0	0	0	0
BREADY	0	0	0	⇄	0	0	0	0

- **T1** AW accepted (slave idle). W is already offered but WREADY is LOW — slave waits for AW first.
- **T2, T3** two beats; WLAST with D1.
- **T4** B handshake → slave back to IDLE, AWREADY HIGH again at T5.

5 Write tests & expected memory (32-bit RAM)

Test	Call	Expect
Single	`addr 0x00, len 0, size 2, INCR`, D = 0xCAFE_F00D	mem[0] = CAFEF00D
INCR ×4	`addr 0x10, len 3`, D = 1,2,3,4	mem[4..7] = 1,2,3,4
WRAP ×4	`addr 0x38, len 3, WRAP`, D = A,B,C,D	mem[14]=A, [15]=B, [12]=C, [13]=D
FIXED ×3	`addr 0x40, len 2, FIXED`, D = X,Y,Z	mem[16] = Z (last wins)
Unaligned	`addr 0x51, len 1`, strb 1110, 1111	mem[20] bytes 1–3, mem[21] all
BREADY delay	hold BREADY LOW 5 cycles	BVALID stays HIGH, no lost response

PARTIAL-STROBE EXAMPLE

mem before	11	22	33	44	
WDATA	AA	BB	CC	DD	WSTRB = 0101
mem after	11	BB	33	DD	= 0x11BB33DD

6 Checking the write

OPTION A · PEEK MEMORY (QUICK)

```
if (tb_top.dut.u_ram.mem[addr >> 2] !== exp)
  $error("mem[%h] = %h, exp %h",
         addr, tb_top.dut.u_ram.mem[addr >> 2], exp);
```

OPTION B · READ BACK OVER AXI (BETTER)

Write, then `axi_read()` the same address and compare — tests the read path too and doesn't depend on internal names.

OPTION C · REFERENCE MODEL

Keep a byte-array model in the TB; apply every write with WSTRB; compare on every read. This is the scoreboard idea, scaled up in UVM.

Always check **BRESP** and **BID** too — a "passing" write with SLVERR is a bug.

! Common testbench mistakes

- Waiting for AWREADY/WREADY **before** raising VALID → can deadlock with some slaves.
- Using blocking `=` on DUT inputs at posedge → race conditions.
- Checking READY with `wait(AWREADY)` (level) instead of at a clock edge → miscounted beats.
- Forgetting WLAST, or setting it on every beat.
- Driving AW and W strictly in sequence only — never exercises W-before-AW.
- Leaving BREADY LOW forever → write never completes.
- Not filling `wdata_q` with LEN+1 entries.
- No timeout → hung simulation instead of a clear failure.
- Checking memory before B handshake has completed.

? Interview Q&A

Question	Answer
Why fork AW and W?	Channels are independent; a master mustn't serialize them.
Why NBA in the TB?	Avoids races with DUT flops at the same edge.
How do you know a beat was taken?	VALID && READY sampled at posedge.
What's an AXI wrapper?	Top that exposes AXI ports around the core.
How to check the write?	Read back / reference model + BRESP.
Next step to UVM?	Task → driver, checks → monitor + scoreboard, tests → sequences.

DV INTERVIEW · GLANCE SHEET 10 · PROJECT PART 3

Reading the RAM Back over AXI

Issue AR, collect LEN+1 beats on R, check RID / RRESP / RLAST on every beat, and compare against what was written.

AR handshake → R beats × (LEN+1)
RLAST only on beat LEN
write → read → compare = self-checking

1 End-to-end flow of the project

1 · Reset — all VALIDs LOW, ARESETn released on an edge

▶

2 · axi_write() — AW ‖ W, then B · BRESP = OKAY

▶

3 · axi_read() — AR, then LEN+1 R beats into a queue

▶

4 · Compare — each beat vs expected (model or write data)

▶

5 · Report — error count → PASS / FAIL

The read must start **after the B handshake** of the write it depends on. AXI gives no ordering between reads and writes, even to the same address — waiting for BRESP is how the master guarantees it reads the new data.

2 The `axi_read` task

```
task automatic axi_read(input  [ID_W-1:0]   id,
                        input  [ADDR_W-1:0] addr,
                        input  [7:0] len, input [2:0] size,
                        input  [1:0] burst,
                        output logic [DATA_W-1:0] rd_q[$]);
  rd_q.delete();
  // ---- read address channel
  ARID <= id;  ARADDR <= addr;  ARLEN <= len;
  ARSIZE <= size;  ARBURST <= burst;  ARVALID <= 1;
  do @(posedge ACLK); while (!ARREADY);
  ARVALID <= 0;

  // ---- read data channel
  for (int i = 0; i <= len; i++) begin
    do begin
      RREADY <= rand_bp ? $urandom_range(0, 1) : 1;
      @(posedge ACLK);
    end while (!(RVALID && RREADY));          // beat accepted
    rd_q.push_back(RDATA);
    if (RID   !== id)       $error("RID %0h != ARID %0h", RID, id);
    if (RRESP !== 2'b00)    $error("RRESP %b beat %0d", RRESP, i);
    if (RLAST !== (i == len)) $error("RLAST wrong at beat %0d", i);
  end
  RREADY <= 0;
endtask
```

- **Every beat** is checked for RID, RRESP and RLAST — RRESP is per beat, unlike BRESP.
- `rand_bp` = 1 randomly drops RREADY to exercise slave back-pressure handling.
- RDATA is captured on the same edge as the handshake, so it's the data the slave was holding.

3 4-beat read with one RREADY stall

`ARADDR 0x10 · ARLEN 3 · ARSIZE 010 · INCR` against the Part-1 slave.

ACLK	T1	T2	T3	T4	T5	T6	T7	T8
ARVALID	⇄	0	0	0	0	0	0	0
ARREADY	⇄	0	0	0	0	0	1	1
RVALID	0	⇄	1	⇄	⇄	⇄	0	0
RREADY	0	⇄	0	⇄	⇄	⇄	0	0
RDATA	–	D0	D1	D1	D2	D3	–	–
RLAST	0	0	0	0	0	1	0	0

- **T1** AR accepted; slave moves to R_DATA.
- **T2** D0 (mem[4]). **T3** TB drops RREADY → slave **holds D1** (address only moves on handshake).
- **T4, T5, T6** D1, D2, D3; RLAST with D3 → slave back to R_IDLE, ARREADY HIGH at T7.

With RREADY always HIGH the same read finishes one cycle earlier (T5). Run both — the stall case is where slave bugs hide.

4 Read-after-write tests & expected data (32-bit RAM)

#	Write	Read	Expected read data
1	INCR 0x10, 4 beats: 1, 2, 3, 4	INCR 0x10, LEN 3	1, 2, 3, 4
2	WRAP 0x38, 4 beats: A, B, C, D	INCR 0x30, LEN 3	**C, D, A, B** (wrap order un-rotated)
3	INCR 0x30, 4 beats: P, Q, R, S	WRAP 0x38, LEN 3	**R, S, P, Q** (rotated on read)
4	FIXED 0x40, 3 beats: X, Y, Z	FIXED 0x40, LEN 2	Z, Z, Z
5	0x11223344, then 0xAABBCCDD with WSTRB 0101	single 0x20	0x11BB33DD
6	INCR 0x00 × 16 beats (pattern i)	INCR 0x00 × 16, random RREADY	0 … 15, every RLAST correct
7	nothing written	any address	X — initialise mem or `$readmemh` first

TEST 1 IN CODE

```
wdata_q = '{32'h1, 32'h2, 32'h3, 32'h4};
wstrb_q = '{4'hF, 4'hF, 4'hF, 4'hF};
axi_write(0, 32'h10, 3, 3'b010, INCR);
axi_read (0, 32'h10, 3, 3'b010, INCR, rd_q);
foreach (rd_q[i])
  if (rd_q[i] !== wdata_q[i]) begin
    $error("beat %0d: got %h exp %h",
           i, rd_q[i], wdata_q[i]);
    errors++;
  end
```

Tests 2 and 3 are the best pair: writing one burst type and reading with another proves the **address generator** is right, not just that data loops back.

5 Narrow & concurrent reads

PICKING BYTES FROM A FULL-WIDTH BEAT

Read 1 byte at 0x22 (SIZE 000). Slave returns the whole word; master takes lane `0x22 % 4 = 2`.

RDATA [11] [BB] [33] [DD]

```
byte_val = RDATA[8*(addr % 4) +: 8];  // = 8'hBB
```

READ DURING A WRITE

```
fork
  axi_write(0, 32'h100, 7, 3'b010, INCR);
  axi_read (1, 32'h200, 7, 3'b010, INCR, rd_q);
join
```

Different addresses, both channels busy at once — proves the read and write FSMs really are independent.

! Read-side mistakes

- Starting the read before the write's **BRESP** → may read stale data.
- Checking RRESP only on the last beat (it's per beat).
- Not checking RLAST position — off-by-one bugs slip through.
- Holding RREADY HIGH forever — never tests slave back-pressure.
- Sampling RDATA without RVALID && RREADY at the same edge.
- Expecting a narrow read to return only the requested bytes — the master must pick lanes.
- Comparing against uninitialised memory (X) and calling it a pass because `!=` returns X — use `!==`.

? Interview Q&A

Question	Answer
How many R beats for ARLEN = 7?	8, RLAST on the 8th.
RRESP vs BRESP?	RRESP every beat; BRESP once per burst.
Why wait for BRESP before reading?	No R/W ordering in AXI; BRESP proves the write landed.
Why use `!==` in checks?	Catches X/Z; `!=` would give X and skip the error.
Best single test for the address logic?	Write with WRAP, read with INCR (and vice versa).
How would this become UVM?	Tasks → driver, checks → monitor + scoreboard with memory model, tests → sequences.

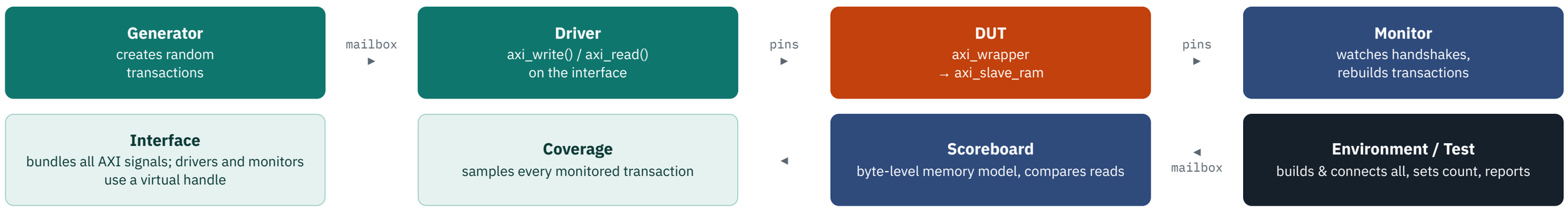
DV INTERVIEW · GLANCE SHEET 11 · PROJECT FINAL PART

Verifying the AXI RAM: Full Testbench

From hand-written tasks to a layered, random, self-checking environment with coverage — the same shape as UVM.

generate → drive → monitor → check
random + constrained + covered
done = 0 errors & 100% coverage

1 Layered testbench architecture



Why layers? Each block has one job, so you can swap the generator (new test) without touching the driver, or reuse the monitor elsewhere.

The monitor is passive — it never drives a pin. The scoreboard trusts the monitor, not the driver, so driver bugs don't hide DUT bugs.

The driver reuses the `axi_write` / `axi_read` tasks from the previous two sheets.

2 Transaction class & constraints

```
class axi_txn;
  rand bit       wr;           // 1 = write, 0 = read
  rand bit [3:0]  id;
  rand bit [31:0] addr;
  rand bit [7:0]  len;
  rand bit [2:0]  size;
  rand bit [1:0]  burst;
  rand bit [31:0] data[];
  rand bit [3:0]  strb[];
       bit [1:0]  resp;         // filled by monitor

  constraint c_legal {
    burst != 2'b11;
    size  <= 3'b010;                    // 32-bit bus
    burst == 2'b10 -> len inside {1, 3, 7, 15};
    burst == 2'b00 -> len < 16;
    burst == 2'b10 -> addr % (1 << size) == 0;
  }
  constraint c_range {
    addr < DEPTH * 4;
    burst == 2'b01 ->
      (addr % 4096) + (len + 1) * (1 << size) <= 4096;
    burst == 2'b01 -> addr + (len + 1) * (1 << size) <= DEPTH * 4;
  }
  constraint c_arrays {
    data.size() == len + 1;
    strb.size() == len + 1;
  }
  constraint c_dist {
    len dist {0 := 20, [1:15] :/ 50, [16:255] :/ 30};
    solve burst before len;
  }
endclass
```

3 Environment & test

```
class axi_env;
  generator  gen;   driver    drv;
  monitor    mon;   scoreboard sb;
  mailbox #(axi_txn) gen2drv = new();
  mailbox #(axi_txn) mon2sb  = new();

  function new(virtual axi_if vif);
    gen = new(gen2drv);
    drv = new(gen2drv, vif);
    mon = new(mon2sb,  vif);
    sb  = new(mon2sb);
  endfunction

  task run(int n);
    gen.count = n;
    fork
      gen.run();  drv.run();
      mon.run();  sb.run();
    join_none
    wait (sb.checked == n);       // every txn seen & checked
    sb.report();                  // PASS only if errors == 0
  endtask
endclass
```

DRIVER LOOP

```
task run();
  forever begin
    gen2drv.get(t);
    if (t.wr) axi_write(t);   // from sheet 9
    else      axi_read (t);   // from sheet 10
  end
endtask
```

End of test = scoreboard has checked N transactions, not "generator finished" — otherwise the last responses are never compared.

4 Monitor

```
task run();
  forever begin
    @(posedge vif.ACLK);
    if (vif.AWVALID && vif.AWREADY)
      wr = capture_aw();            // addr, len…
    if (vif.WVALID && vif.WREADY) begin
      wr.data.push_back(vif.WDATA);
      wr.strb.push_back(vif.WSTRB);
    end
    if (vif.BVALID && vif.BREADY) begin
      wr.resp = vif.BRESP;
      mon2sb.put(wr);               // write complete
    end
    // same pattern for AR / R → put on RLAST
  end
endtask
```

- Samples **only on handshake edges**.
- Sends a write only after B, a read only after RLAST.
- Also a good home for protocol checks (or bind SVA).

5 Scoreboard

```
bit [7:0] mem_model [bit [31:0]];   // sparse bytes

task run();
  forever begin
    mon2sb.get(t);
    a = t.addr;
    for (int b = 0; b <= t.len; b++) begin
      for (int l = 0; l < 4; l++) begin
        ba = (a & ~32'h3) + l;          // byte address
        if (!in_beat(a, t.size, l)) continue;
        if (t.wr && t.strb[b][l])
          mem_model[ba] = t.data[b][8*l +: 8];
        else if (!t.wr && mem_model.exists(ba) &&
                 t.data[b][8*l +: 8] !== mem_model[ba])
          error(t, b, l);
      end
      a = next_addr(a, t.len, t.size, t.burst);
    end
    checked++;
  end
endtask
```

Byte-level model handles narrow, unaligned and partial-strobe writes for free. `next_addr()` is the same function as the RTL — but write it **independently** in the TB, or a shared bug passes.

6 Functional coverage

```
covergroup axi_cg with function sample(axi_txn t);
  cp_dir   : coverpoint t.wr;
  cp_burst : coverpoint t.burst
             { bins b[] = {0, 1, 2}; }
  cp_len   : coverpoint t.len {
    bins one     = {0};
    bins short_  = {[1:15]};
    bins sixteen = {15};
    bins long_   = {[16:254]};
    bins max     = {255}; }
  cp_size  : coverpoint t.size { bins s[] = {[0:2]}; }
  cp_align : coverpoint (t.addr % (1 << t.size)) == 0);
  cp_strb  : coverpoint t.strb[0] {
    bins full = {4'hF}; bins none = {4'h0};
    bins part = default; }
  x_bls    : cross cp_burst, cp_len, cp_size;
  x_dir    : cross cp_dir, cp_burst;
endgroup
```

Plus **protocol coverage** sampled in the monitor: READY back-pressure on each channel, W-before-AW, read and write overlapping.

7 Test plan

Test	How	Targets
sanity	Directed single write + read	Bring-up, connectivity
burst_types	Each of FIXED / INCR / WRAP, write then read	Address generator
cross_burst	Write WRAP, read INCR and vice versa	Address order bugs
strobe	Random WSTRB, narrow & unaligned	Byte enables, lanes
backpressure	Random BREADY / RREADY delays	Hold & stable rules
concurrent	Write and read threads in parallel	FSM independence
max_len	LEN 255 INCR at a 4 KB-aligned base	Counters, RLAST
random	N = 10 000 random txns, many seeds	Everything; closes coverage
reset	Assert ARESETn mid-burst, then resume	Reset recovery
error	Out-of-range address, BURST = 11	SLVERR handling

8 Bugs this TB is built to catch

Bug	Caught by
WRAP wraps at wrong boundary	cross_burst + scoreboard
Beat-2 address not aligned	strobe (unaligned) + scoreboard
WSTRB ignored	strobe + byte model
RLAST off by one	axi_read RLAST check
RDATA changes while stalled	backpressure + SVA
BID / RID wrong	Driver ID checks
Read blocks while writing	concurrent (timeout)
Counter overflow at LEN 255	max_len
Stuck after reset	reset test

✓ Sign-off checklist

- [ ] All tests pass on many seeds, **0 errors**
- [ ] **Functional coverage 100%** of the plan (or waivers justified)
- [ ] **Code coverage** reviewed: line, branch, toggle, FSM states & transitions
- [ ] All **SVA** protocol assertions enabled and passing
- [ ] No X on outputs after reset
- [ ] Every bug filed, fixed and re-run in regression
- [ ] Scoreboard actually compared N transactions (no silent pass)

100% coverage with 0 checks is a classic trap — always confirm the checker ran.

U Same TB in UVM

This TB	UVM
axi_txn	uvm_sequence_item
generator	uvm_sequence + uvm_sequencer
driver	uvm_driver
monitor	uvm_monitor + analysis port
mailbox	TLM port / analysis FIFO
scoreboard	uvm_scoreboard
driver + monitor + sqr	uvm_agent
env	uvm_env
test / N	uvm_test + config_db
wait(checked == n)	objections

? Interview Q&A

Question	Answer
Why a monitor if the driver knows what it sent?	Driver could be wrong; monitor shows what really happened on the pins.
Why a byte-level model?	Handles strobes, narrow and unaligned without special cases.
How do you know you're done?	Coverage closed, 0 errors over seeds, code coverage reviewed.
Random or directed?	Directed for bring-up, constrained random to close coverage.
Biggest risk in this TB?	Sharing address logic with RTL — common-mode bugs pass.
How to end the test cleanly?	Wait for scoreboard count (UVM: objections).

PROJECT SHEET 1 · NITISHVERIFIES / AXI4_UVM_VERIFICATION

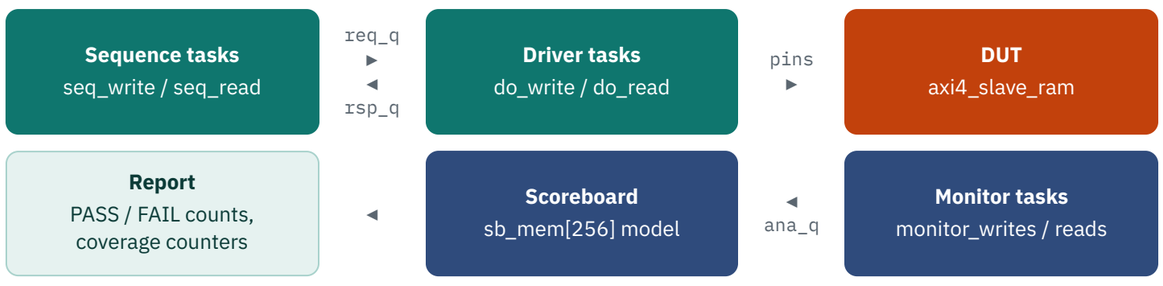
Your AXI4-Lite UVM Project, Explained

Know every file well enough to defend it. Interviewers at top silicon companies go deep on the project you list on your resume.

DUT: AXI4-Lite slave, 256 × 32-bit RAM
TB: task-based (runs on iverilog)
+ full UVM class library (tb/*.sv)

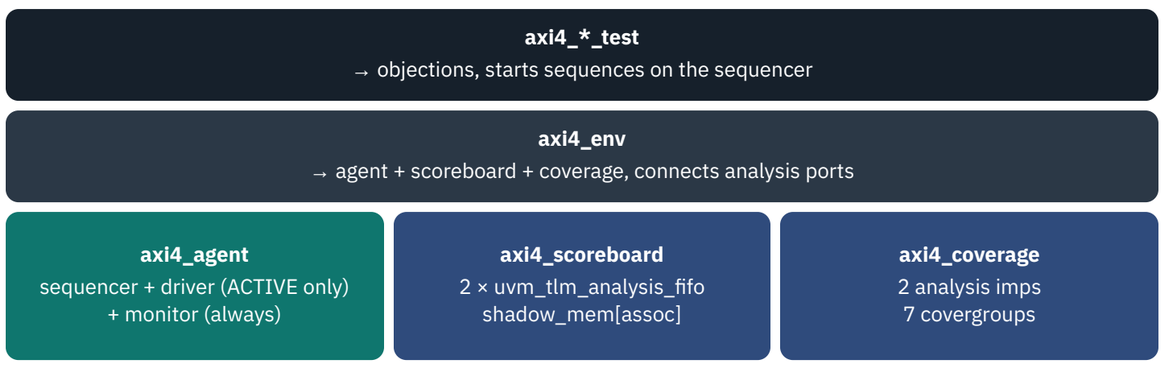
1 The repo has two testbenches — know which one actually runs

A · RUNNABLE: AXI4_TB_TOP.SV (TASK-BASED, IVERILOG)



- Hand-rolled FIFOs (flat arrays + head/tail/count + `wait(cnt>0)`) replace `mailbox`.
- Test picked by a compile-time `` `define `` from the Makefile.
- This is what `make all_tests` builds and runs.

B · UVM LIBRARY: TB/AXI4_*.SV (FOR VCS / QUESTA / XCELIUM)



Know this before you're asked: the Makefile never compiles these files. They are UVM-style code that hasn't been run yet — see the bugs sheet. Say so honestly, or fix it first.

2 UVM concept → where it lives in your repo

UVM concept	Task-based TB (A)	UVM library (B)
Transaction	`axi4_pkg::axi4_seq_item` (plain class)	`axi4_seq_item` extends uvm_sequence_item, rand + constraints, do_copy / do_compare
Sequence	seq_write / seq_read tasks	9 sequences: single, burst, rw, stress, error, wr_rd_check
Seq ↔ driver	req_q / rsp_q FIFOs	start_item / finish_item ↔ get_next_item / item_done
Driver	do_write: AW & W together, then B	Same, through `vif.driver_cb`
Monitor	Wait AW → W → B handshakes, push to ana_q	Forked write/read tasks → `ap_write`, `ap_read`
Analysis path	ana_q FIFO	analysis port → `uvm_tlm_analysis_fifo` (SB) and analysis imps (coverage)
Scoreboard	sb_mem[256], strobe-aware	assoc array shadow_mem, expects SLVERR ≥ 256 words
Coverage	Write / read counters	Address quadrant, data patterns, WSTRB, BRESP/RRESP
Config / VIF	Direct hierarchical access	`uvm_config_db::get(…"AXI4_VIF"…)`
Factory	—	`type_id::create` everywhere
End of test	`repeat(20) @(posedge)` + $finish	raise / drop objection + fixed #delay
Active / passive	—	`get_is_active()` in agent build_phase

3 The DUT: axi4_slave_ram

WRITE FSM (AS SHIPPED)



READ FSM



- **Single outstanding** per direction; reads and writes are independent FSMs.
- `apply_strobe()` merges new bytes into the old word using WSTRB.
- Index = `(addr >> 2) % 256` — out-of-range addresses **wrap** (alias), no SLVERR.
- RDATA is **registered**: it's stable while RVALID is held, which satisfies the handshake rule.
- Memory initialised to 0 in an `initial` block (simulation only — not reset logic).

4 Stimulus: the seq_item constraints

Constraint	What it does	Interview angle
`c_txn_type_dist`	60% writes, 40% reads	Why `:=` vs `:/` in dist
`c_addr_legal`	0x000–0x3FC, word aligned	How the error sequence overrides it with inline `with`
`c_wstrb`	Non-zero; 70% 1111, 5% each of 6 partial patterns	Only 7 of 15 legal patterns can ever occur
`c_prot`	Always 000	Hard constraint = no PROT coverage

Inline override: `axi4_error_seq` randomises with `addr inside {[0x400:0xFFFC]}` — but `c_addr_legal` still forbids that, so `randomize()` fails. Correct way: `txn.c_addr_legal.constraint_mode(0)` first, or put the range in a soft constraint.

5 Tests and what each really proves

Test	Stimulus	Really proves
write_test	8 writes, addr 0x00–0x1C	BRESP OKAY only — data not read back
read_test	8 reads after reset	Reset value 0
rw_test	Write then read, 8 addresses	Full-word data path
stress_test	32 random R/W, random strobes 1–15	Strobe merge + mixed traffic
error_test	Write/read 0xFFFF_FFFC	Nothing checked — aliasing hides it
wr_rd_chk_test	DEADBEEF, then CAFE0000 strb 1100	Expects 0xCAFE_BEEF

Every test drives AW and W in the **same cycle** with READY always accepted promptly. Real masters don't — that's the gap the bugs sheet exploits.

6 60-second project pitch

"I verified an **AXI4-Lite slave RAM** — 256 × 32 with byte strobes — using a **UVM-structured, self-checking** testbench: agent with driver, monitor and sequencer, a **scoreboard with a strobe-aware reference model**, and **functional coverage** on addresses, strobes and responses.

When I stressed the handshake ordering I found the DUT **deadlocks if write data arrives before the address**, that it **aliases out-of-range addresses** instead of returning SLVERR, and that my regression could **report PASS while the scoreboard logged errors**. I fixed all three and added assertions so they can't come back."

Only say the second paragraph **after** you've applied the fixes from the next sheet. Then it's true, and it's your strongest story.

7 Trace one write end to end

1. Test calls `seq_write(0x10, D)` → item pushed to `req_q`, sequence blocks on `rsp_q`.
2. Driver pops it, drives AWADDR/AWVALID + WDATA/WSTRB/WVALID on the next edge.
3. Slave (WR_ADDR) sees both handshakes in one cycle, writes `mem[4]`, raises BVALID.
4. Driver raises BREADY, captures BRESP, puts the item on `rsp_q` → sequence unblocks.
5. Monitor independently saw AW, W, B handshakes → builds its own item → `ana_q`.
6. Scoreboard applies strobes to `sb_mem[4]`, checks BRESP.

Key point: the scoreboard uses the **monitor's** item, never the driver's — so a driver bug can't hide a DUT bug.

8 Run it

```
sudo apt install iverilog gtkwave
cd sim
make sim TEST=axi4_rw_test SIM=iverilog
make all_tests SIM=iverilog
make waves TEST=axi4_rw_test
```

MAKEFILE ALSO TARGETS

- **Questa** — `vlog -uvm`, `vsim +UVM_TESTNAME`, UCDB coverage merge
- **VCS** — `-ntb_opts uvm`, `-cm line+cond+fsm+branch+tgl`, urg
- **Xcelium** — `xrun -uvm -coverage all`, imc
- Falls back to iverilog when the commercial tool isn't found.

Be able to explain `-sv_seed` / `+ntb_random_seed`: same seed reproduces the same random run — the basis of debugging a regression failure.

PROJECT SHEET 2 · EVERY FINDING REPRODUCED IN SIMULATION

Bugs in the Repo — Evidence & Fixes

Ran `make all_tests SIM=iverilog` on the repo as-is, then directed probes. Fixes below were re-run: 6/6 pass for real, and failures now fail.

DUT design bug
TB testbench bug
TOOL simulator limitation

#1 Regression says PASS while the scoreboard logs FAILs

TB · most important

Evidence (repo as-is): `ALL TESTS DONE: PASS=6 FAIL=0` — yet the logs show scoreboard failures in 5 of 6 tests: read 7/1, rw 8/8, stress 30/2, error 1/1, wr_rd_chk 2/1 (pass/fail).

Root cause: `scoreboard_report()` only `$display`s "TEST FAILED"; `vvp` still exits 0, and the Makefile judges a test by exit code. So **this testbench cannot fail**.

Interview gold: "How do you know your test passed?" Answer: a non-zero exit / UVM_ERROR count checked by the regression script, and a check that the scoreboard compared **more than zero** items.

FIX (VERIFIED: FAILING RUN NOW EXITS 1 → MAKE REPORTS FAILED)

```
if (sb_fail == 0 && sb_pass > 0)
  $display("[SB] *** TEST PASSED ***");
else begin
  $display("[SB] *** TEST FAILED ***");
  $fatal(1, "scoreboard failures");   // non-zero exit
end
// UVM: fail in report_phase if UVM_ERROR count > 0,
// and grep the log for "UVM_ERROR :    0" in regression
```

#2 Deadlock when write data arrives before the address

DUT · protocol violation

In `WR_ADDR` the slave holds **WREADY high** but only looks at AWVALID. A master that sends W first completes the W handshake — the slave **drops the data**. When AW arrives the FSM goes to `WR_DATA` and waits forever for a W beat that already happened.

Evidence: directed probe (W first, then AW, BREADY high) → `NO BVALID after 50 cycles`.

Why the TB missed it: the driver always raises AWVALID and WVALID in the **same cycle**. W-before-AW is legal AXI and the stimulus never produced it — a **stimulus coverage hole**, not a checker problem.

FIX: ACCEPT AW AND W INDEPENDENTLY (VERIFIED: BVALID ARRIVES)

```
assign awready = !aw_got && !bvalid;   // from own state only
assign wready  = !w_got  && !bvalid;
always_ff @(posedge aclk or negedge aresetn)
  if (!aresetn) {aw_got, w_got, bvalid} <= '0;
  else begin
    if (awvalid && awready) begin aw_addr <= awaddr; aw_got <= 1; end
    if (wvalid  && wready)  begin w_data <= wdata; w_strb <= wstrb; w_got <= 1; end
    if (aw_got && w_got && !bvalid) begin
      if (in_range) mem[idx] <= apply_strobe(mem[idx], w_data, w_strb);
      bresp  <= in_range ? 2'b00 : 2'b10;     // also fixes #3
      bvalid <= 1;  aw_got <= 0;  w_got <= 0;
    end
    if (bvalid && bready) bvalid <= 0;
  end
```

#3 No SLVERR — out-of-range addresses alias

DUT

The header comment and the UVM scoreboard both expect **SLVERR** above 256 words, but the RTL indexes with `(addr >> 2) % 256` and always returns OKAY.

Evidence: write DEADBEEF to `0x400` → `BRESP = 00`; then read `0x000` → `DEADBEEF`. The out-of-range write **corrupted word 0**.

error_test "passed" because it wrote and read the same aliased address — a **self-consistent but wrong** result. Classic reason to check against a spec-based model, not a DUT echo.

Fix: `in_range = addr[31:2] < MEM_DEPTH`. Write: skip the memory update, BRESP = 10. Read: RDATA = 0, RRESP = 10. Scoreboard expects SLVERR and doesn't update the model. Verified: both return `10`, word 0 untouched.

#4 Monitor records a bogus X transaction at start-up

TB

Monitor loops on `while (!(bus.arvalid && bus.arready))`. At start ARVALID is **X**, ARREADY is 1 → condition is X → a `while` treats X as false and **exits immediately**, capturing a read with `addr = X`.

Evidence: `[SB] FAIL READ addr=0xxxxxxxxx` as the first scoreboard entry in 5 tests.

Fix (verified): 4-state compare `while (!(bus.arvalid === 1'b1 && bus.arready === 1'b1))`, and drive all master VALIDs to 0 from time 0, not after reset.

Concept: X-pessimism / X-optimism. `if` and `while` treat X as false; `==` returns X; `===` returns 0/1. Add an SVA "no X on VALID after reset".

#5 Scoreboard model corrupts data on iverilog

TOOL

`sb_mem[idx][b*8+:8] = tr.data[b*8+:8]` — a variable part-select **on a class property**. iverilog returns the low byte for every lane.

Evidence: 5-line repro: `c.data = 40895E81` → model got `81818181`. Same pattern in rw_test: `exp=0x81818181 got=0x40895e81` — the DUT was right, the **checker** was wrong.

Fix (verified): copy into locals first: `d = tr.data; st = tr.strb;` then part-select `d`.

Lesson: prove the checker can catch a **known-bad DUT**, and when a mismatch appears, suspect the model as much as the RTL.

#6 The UVM files have never been compiled

TB

Problem	Effect
Makefile compiles only pkg + interface + tb_top	tb/axi4_agent, driver, env... are dead code
No `import uvm_pkg::*`, `run_test()` or `config_db::set`	Nothing would start; driver's `get` would hit uvm_fatal
Interface has no `driver_cb` / `monitor_cb`, no modports, no SVA (header claims all three)	Driver and monitor don't compile
`uvm_analysis_imp_write` used without `` `uvm_analysis_imp_decl(_write) ``	Coverage class doesn't compile
Typo `bins55_pat` (missing space)	Syntax error in cg_wr_data
Two different `axi4_seq_item` classes	Name clash if both compiled
error_seq inline range conflicts with `c_addr_legal`	randomize() fails → uvm_fatal

#7 Coverage & stimulus gaps

TB

- **WSTRB:** comment says "all 15 patterns", covergroup has 10 bins, constraint can only generate 7 → 0010, 0100, 1000, 0111, 1110 are **unreachable**; coverage can never close.
- **No protocol coverage:** handshake order (AW first / W first / same cycle) and back-pressure length are never measured — exactly where bug #2 hid.
- **READY never randomised:** driver raises BREADY / RREADY at once; slave hold behaviour is barely exercised.
- **SLVERR bins** unreachable while #3 exists — a coverage hole that would have flagged the bug if anyone had looked.
- **UVM scoreboard skips** reads of never-written words — but the RAM resets to 0, so those could be checked.
- **End of test** by fixed `#100ns` instead of waiting for the scoreboard to drain.

Coverage only tells you what you **asked** about. A 100% number on a weak plan proves little.

★ Turn this into your STAR story

SITUATION

AXI4-Lite slave RAM, self-checking TB, regression green 6/6.

TASK

Make sure green actually meant correct before calling it done.

ACTION

- Read the logs, not the summary → found FAIL lines hidden behind exit code 0.
- Root-caused each: X-pessimism in the monitor, a simulator part-select bug in the model.
- Added W-before-AW stimulus → **DUT deadlock**; probed out-of-range → **aliasing, no SLVERR**.
- Fixed RTL, made failures fatal, added SVA + handshake-order coverage.

RESULT

2 real DUT bugs fixed, a regression that can fail, and coverage that measures the protocol, not just addresses.

Do the fixes in the repo first — then every line of this story is true and you can show the commits.

PROJECT SHEET 3 · FROM "STUDENT PROJECT" TO "HIRE SIGNAL"

Upgrade Roadmap for Top-Tier DV Interviews

What to add to the repo, in priority order, and what big silicon DV interview loops commonly test.

P0 correctness — do first
P1 strongest interview signal
P2 industry methodology
P3 stand-out extras

1 Roadmap

Pri	Add	What it demonstrates	Interview question it prepares you for
P0	Make failures fatal; check pass count > 0	You trust nothing that can't fail	"How does your regression decide pass/fail?"
P0	Fix W-before-AW deadlock + SLVERR (bugs #2, #3)	Protocol knowledge, real bug-finding	"Tell me about a bug you found."
P0	Get the UVM layer compiling and running on a real simulator (EDA Playground has free UVM)	Your resume claim is true	"Walk me through your env's build/connect phases."
P1	SVA protocol checker, bound to the DUT	Assertions, bind, temporal logic	"Write an assertion for the VALID/READY rule."
P1	Driver knobs: random AW/W skew, BREADY/RREADY back-pressure (config object)	Stimulus that hunts corner cases	"How would you find a deadlock?"
P1	Protocol coverage: handshake order, stall lengths, resp × direction, all 15 strobes	Coverage plan driven by risk, not by fields	"You're at 100% coverage but still found a bug. Why?"
P2	UVM RAL model + AXI-Lite adapter + predictor	AXI-Lite is a register bus — RAL is how industry tests it	"Frontdoor vs backdoor? Explicit vs implicit prediction?"
P2	Regression script: N seeds, log parsing, merged coverage, failure triage	Scale and automation (Python/Make)	"Test fails 1 in 1000 seeds. What do you do?"
P3	Formal: prove the SVA properties (e.g. SymbiYosys free, or JasperGold/VC Formal at work)	Formal mindset — growing fast in CPU/GPU teams	"Which blocks suit formal, and why?"
P3	Full AXI4 slave: bursts, IDs, outstanding (sheets 5–11 of this canvas)	Real protocol complexity	"Scoreboard with out-of-order responses?"

2 P1 · AXI4-Lite SVA checker (bind it, don't edit the RTL)

```
module axi_lite_sva (input logic aclk, aresetn,
  input logic awvalid, awready, wvalid, wready, bvalid, bready,
  input logic arvalid, arready, rvalid, rready,
  input logic [31:0] awaddr, wdata, rdata, input logic [3:0] wstrb,
  input logic [1:0]  bresp, rresp);

  default clocking cb @(posedge aclk); endclocking
  default disable iff (!aresetn);

  // VALID held + payload stable until READY (all 5 channels)
  a_aw: assert property (awvalid && !awready |=> awvalid && $stable(awaddr));
  a_w : assert property (wvalid && !wready |=> wvalid && $stable({wdata, wstrb}));
  a_b : assert property (bvalid && !bready |=> bvalid && $stable(bresp));
  a_r : assert property (rvalid && !rready |=> rvalid && $stable({rdata, rresp}));
  a_x : assert property (!$isunknown({awvalid, wvalid, bvalid, arvalid, rvalid}));

  // B only after both AW and W were accepted; then within 8 cycles
  logic [7:0] aw_n, w_n, b_n;
  always_ff @(posedge aclk or negedge aresetn)
    if (!aresetn) {aw_n, w_n, b_n} <= '0;
    else begin
      aw_n <= aw_n + (awvalid && awready);
      w_n  <= w_n  + (wvalid  && wready);
      b_n  <= b_n  + (bvalid  && bready);
    end
  a_b_order: assert property (bvalid |-> aw_n != b_n && w_n != b_n);
  a_b_live : assert property (aw_n != b_n && w_n != b_n |-> ##[0:8] bvalid);
endmodule

bind axi4_slave_ram axi_lite_sva u_sva (.*);   // port names match
```

`a_b_live` fires on the original RTL in the W-before-AW case — the assertion would have caught bug #2 on day one.

3 P1 · Driver knobs

```
class axi_cfg extends uvm_object;
  `uvm_object_utils(axi_cfg)
  rand int unsigned max_skew  = 4;  // AW vs W start delay
  rand int unsigned max_bstall = 6;  // BREADY low cycles
  rand int unsigned max_rstall = 6;  // RREADY low cycles
  function new(string name = "axi_cfg"); super.new(name); endfunction
endclass

// in drive_write(): channels start independently
fork
  begin repeat ($urandom_range(0, cfg.max_skew)) @(vif.driver_cb);
        drive_aw(txn); end
  begin repeat ($urandom_range(0, cfg.max_skew)) @(vif.driver_cb);
        drive_w(txn);  end
join
repeat ($urandom_range(0, cfg.max_bstall)) @(vif.driver_cb);
vif.driver_cb.bready <= 1;
```

Set via `uvm_config_db#(axi_cfg)::set` from the test, so one test can be "no stalls" and another "heavy back-pressure" without touching the driver.

4 P1 · Protocol coverage (sample in the monitor)

```
typedef enum {AW_FIRST, W_FIRST, SAME_CYCLE} order_e;
covergroup cg_proto with function sample(order_e ord,
                        int bstall, int rstall, bit wr, bit [1:0] resp);
  cp_ord : coverpoint ord;
  cp_bst : coverpoint bstall { bins none = {0}; bins short_ = {[1:3]};
                               bins long_ = {[4:$]}; }
  cp_rst : coverpoint rstall { bins none = {0}; bins short_ = {[1:3]};
                               bins long_ = {[4:$]}; }
  cp_resp: coverpoint resp   { bins okay = {0}; bins slverr = {2}; }
  x_ord_bst : cross cp_ord, cp_bst;
  x_dir_resp: cross wr, cp_resp;
endgroup
```

5 P2 · RAL — the standard way to verify an AXI-Lite register/memory block

```
class axi_lite_adapter extends uvm_reg_adapter;
  `uvm_object_utils(axi_lite_adapter)
  function new(string name = "axi_lite_adapter"); super.new(name); endfunction

  virtual function uvm_sequence_item reg2bus(const ref uvm_reg_bus_op rw);
    axi4_seq_item t = axi4_seq_item::type_id::create("t");
    t.txn_type = (rw.kind == UVM_WRITE) ? axi4_seq_item::AXI4_WRITE
                                        : axi4_seq_item::AXI4_READ;
    t.addr = rw.addr;  t.data = rw.data;  t.wstrb = 4'hF;
    return t;
  endfunction

  virtual function void bus2reg(uvm_sequence_item item, ref uvm_reg_bus_op rw);
    axi4_seq_item t;
    if (!$cast(t, item)) `uvm_fatal("ADPT", "bad item type")
    rw.kind   = (t.txn_type == axi4_seq_item::AXI4_WRITE) ? UVM_WRITE : UVM_READ;
    rw.addr   = t.addr;
    rw.data   = (rw.kind == UVM_READ) ? t.rdata : t.data;
    rw.status = (((rw.kind == UVM_WRITE) ? t.bresp : t.rresp) == 2'b00)
                ? UVM_IS_OK : UVM_NOT_OK;
  endfunction
endclass
```

- **Model:** `uvm_reg_block` with a `uvm_mem` (256 × 32) — or a few `uvm_reg`s if you add CSRs.
- **Connect:** `map.set_sequencer(agent.sequencer_h, adapter)`.
- **Predictor:** `uvm_reg_predictor` fed by the monitor → **explicit prediction**, mirror tracks bus traffic from any master.
- **Built-in sequences:** `uvm_reg_hw_reset_seq`, `uvm_reg_bit_bash_seq`, `uvm_mem_walk_seq` — free tests.
- **Backdoor:** `add_hdl_path` → peek/poke the RAM without bus traffic, useful to preload or to check.

Know cold: `write/read` (bus) vs `set/get` (desired value) vs `update/mirror` vs `peek/poke` (backdoor).

6 What big silicon DV interview loops commonly cover

Based on widely shared candidate experiences; exact rounds vary by team and level — ask your recruiter for the format.

SystemVerilog & UVM depth

OOP (virtual methods, polymorphism, deep vs shallow copy), constraints & `solve…before`, coverage, factory, config_db, phases, TLM, sequences.

Assertions & formal

Write SVA live: handshakes, one-hot, req→ack within N, FIFO never overflows. When formal beats simulation.

"Verify this block"

Async FIFO, arbiter, cache, AXI slave, credit-based link. Testplan → checker → coverage → corner cases.

Architecture & protocols

AXI/AHB/APB, caches & coherency (MESI/MOESI), memory ordering, pipelines and hazards, CDC/RDC. Especially deep at CPU/GPU companies.

Debug scenarios

Fails in batch not GUI; 1-in-1000 seed; X propagation; race conditions; "scoreboard mismatch — DUT or model?"

Coding

C/C++ or Python data-structure problems (some loops include a general software round), plus SV coding: constraints, a small scoreboard, a reference model.

Project deep-dive is near-universal: expect 20–40 minutes on one resume project, with "why" after every answer. This canvas's bugs sheet is your preparation for that round.

2-week plan: days 1–3 P0 fixes + push; 4–6 SVA + knobs + coverage; 7–9 RAL; 10–11 regression script; 12–14 mock interviews on sheets 1–4 of this row.

PROJECT SHEET 4 · TIED TO YOUR REPO'S CODE

UVM Core + Project Deep-Dive Q&A

The UVM mechanics your repo uses, explained at the level a senior interviewer pushes to — then the questions they'll ask about your code.

1 Phases

build	connect	end_of_elab	start_of_sim	run	extract	check	report	final
top-down	bottom-up	print topology	banners	task · all in parallel		SB final checks	summary	

Why build is top-down: a parent must exist and set config before children are created. **Connect is bottom-up:** children's ports must exist first.

Only run (and its 12 sub-phases: reset, configure, main, shutdown + pre/post) consumes time. All others are functions.

Your repo: SB checks in `check_phase`, coverage prints in `report_phase`, base test prints topology in `end_of_elaboration_phase` — all correct placements.

2 Factory

```
`uvm_component_utils(axi4_driver)   // register
drv = axi4_driver::type_id::create("drv", this);

// in a test — swap without editing env code
axi4_driver::type_id::set_type_override(
    axi4_err_driver::get_type());
// or only one instance
axi4_driver::type_id::set_inst_override(
    axi4_err_driver::get_type(),
    "env_h.agent_h.driver_h", this);
```

- Calling `new()` directly bypasses overrides — the #1 factory mistake.
- Override must be set **before** the target is created (test's build_phase, before `super` builds env).
- Debug: `factory.print()`.

3 config_db

```
// tb_top (missing in your repo!)
uvm_config_db#(virtual axi4_if)::set(
    null, "uvm_test_top.*", "AXI4_VIF", bus);
run_test();

// driver / monitor build_phase
if (!uvm_config_db#(virtual axi4_if)::get(
        this, "", "AXI4_VIF", vif))
  `uvm_fatal("NOVIF", "vif not set")
```

- Key = context + instance path + field name + **type**. A type mismatch silently fails the get.
- During build, a set from **higher in the hierarchy wins**; at the same level, last set wins.
- Debug: `+UVM_CONFIG_DB_TRACE`.

4 Objections & end of test

```
task run_phase(uvm_phase phase);
  phase.raise_objection(this);
  seq.start(env_h.agent_h.sequencer_h);
  phase.drop_objection(this);
endtask

// instead of the repo's fixed #100ns:
function void end_of_elaboration_phase(uvm_phase p);
  uvm_phase run = uvm_run_phase::get();
  run.get_objection().set_drain_time(this, 200ns);
endfunction
```

- run ends when every objection is dropped (plus drain time).
- Drain time lets the last responses reach the scoreboard.
- Better still: SB raises an objection while it has pending expected items.

5 TLM — the paths in your env

Piece	Semantics	In your repo
analysis port	1 → many, `write()` is a non-blocking **function**	`monitor.ap_write / ap_read`
uvm_tlm_analysis_fifo	Unbounded buffer; subscriber does blocking `get()` in its own thread	Scoreboard `wr_fifo / rd_fifo`
analysis imp	Calls the owner's `write()` immediately	Coverage `wr_export / rd_export`
`` `uvm_analysis_imp_decl(_x) ``	Creates `uvm_analysis_imp_x` → method `write_x()`; needed for >1 imp in one class	**Missing** — add `(_write)` and `(_read)`
seq_item_port / export	Pull interface driver ↔ sequencer	agent connect_phase

FIFO vs imp: the scoreboard needs to *wait* and match (thread + blocking get → FIFO). Coverage just samples on arrival (function → imp).

6 Sequence ↔ driver handshake

```
// sequence body()                  // driver run_phase
start_item(txn);       // wait grant   seq_item_port.get_next_item(txn);
txn.randomize() with {...};          drive(txn);
finish_item(txn);      // wait done   seq_item_port.item_done();
// optional response path
get_response(rsp);                     rsp.set_id_info(txn);
                                       seq_item_port.put_response(rsp);
```

- **Randomize between start_item and finish_item** — late randomization can react to sequencer state.
- `get_next_item` + `item_done` vs `get` (implicit item_done) vs `try_next_item` (non-blocking, for idle cycles).
- Multiple sequences on one sequencer → **arbitration** (FIFO, weighted, random, strict…); `lock` / `grab` for exclusive access.
- **Virtual sequence** on a virtual sequencer coordinates several agents; `` `uvm_declare_p_sequencer `` gives typed access.
- Your repo's driver writes `bresp / rdata` back into the same item — works because the sequence holds that handle after finish_item.

7 object vs component

	uvm_object	uvm_component
Examples	seq_item, sequence, config	driver, monitor, env
Lifetime	Created & dropped any time	Built once, lives all sim
Phases / hierarchy	No	Yes (parent, path)
Macro	uvm_object_utils	uvm_component_utils

Your item uses hand-written `do_copy / do_compare / do_print` instead of field macros — faster and easier to debug; say that's deliberate.

8 Interface, clocking block, modport

```
clocking driver_cb @(posedge aclk);
  default input #1step output #1;
  output awaddr, awvalid, wdata, wstrb,
         wvalid, bready, araddr, arvalid, rready;
  input  awready, wready, bvalid, bresp,
         arready, rvalid, rdata, rresp;
endclocking
clocking monitor_cb @(posedge aclk);
  default input #1step;
  input awaddr, awvalid, awready, /* … all */;
endclocking
modport DRV (clocking driver_cb, input aresetn);
modport MON (clocking monitor_cb, input aresetn);
```

Your driver/monitor reference these, but the interface doesn't define them yet — this is the block to add.

9 Other must-knows

- **Callbacks:** hook into a driver (inject errors) without subclassing.
- **Report catcher:** demote an expected UVM_ERROR in a negative test (e.g. SLVERR test).
- **Verbosity per component:** `+uvm_set_verbosity=…` to debug one block.
- **Transaction recording** to see items on the waveform.
- **uvm_event / uvm_barrier** for cross-component sync.
- **Agent active/passive:** `is_active` via config_db; passive agents reuse your monitor at SoC level.

? Questions an interviewer will ask about *your* repo

Question	Strong answer
Why does the monitor `create()` a new item every loop?	Analysis ports pass handles; reusing one object would overwrite items the SB hasn't processed yet.
Why separate write and read analysis ports?	Independent checker threads per direction; could merge with txn_type, but separate keeps each checker simple.
Why tlm_analysis_fifo in the SB but imps in coverage?	SB waits/blocks per thread; coverage just samples in a function.
When would you make the agent passive?	When another master drives the bus (SoC level) — reuse the monitor, SB and coverage.
Why does `do_compare` skip rdata/resp?	Those are outputs; correctness of outputs is the scoreboard's job against the model.
Why use a clocking block?	Removes TB/DUT races: inputs sampled just before the edge, outputs driven just after.
The SB skips never-written words. Risk?	Reads after reset go unchecked; the RAM resets to 0, so model that and check.

Question	Strong answer
What did your coverage miss?	Handshake order and stalls — exactly where the W-before-AW deadlock hid. Then show the new protocol covergroup.
How would you verify 4 outstanding transactions?	AXI-Lite slave here allows 1; for AXI4 use per-ID queues in the SB (sheet 7) and an outstanding-depth coverpoint.
A test fails on seed 1234 only. Steps?	Rerun that seed, raise verbosity on the agent, dump waves around the first error, check whether DUT or model is wrong, add a directed test.
How do you make sure the checker works?	Inject a known bug (e.g. ignore WSTRB) and confirm the SB fails — mutation testing.
How would you add RAL?	Reg block + adapter (reg2bus/bus2reg) + predictor on the monitor; then built-in reset/bit-bash/mem-walk sequences.
Why iverilog — and its limits?	Free and fast to share; no full UVM, covergroups or constraints, so the runnable TB mirrors UVM with tasks. The UVM layer targets commercial simulators.
What would you do with one more week?	SVA bind + formal proof of the handshake properties, then extend to full AXI4 with bursts and IDs.